



CS4001NT Programming

60% Individual Coursework

Autumn 2024/25

Student Name: Manoj Katuwal

London Met ID: 24045922

College ID: np05cp4a240306

Assignment Due Date: Friday, May 16, 2025

Assignment Submission Date: Friday, May 16, 2025

I confirm that I understand my coursework needs to be submitted online via MySecondTeacher under the relevant module page before the deadline in order for my assignment to be accepted and marked. I am fully aware that late submissions will be treated as non-submission and a marks of zero will be awarded.

CS4001NT Programming

 Islington College, Nepal

Document Details

Submission ID

trnoid::3618:96176111

Submission Date

May 16, 2025, 10:36 AM GMT+5:45

Download Date

May 16, 2025, 10:41 AM GMT+5:45

File Name

CS4001NT Programming

File Size

45.1 KB

59 Pages

6,513 Words

38,528 Characters

12% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

Match Groups

-  **65 Not Cited or Quoted 10%**
Matches with neither in-text citation nor quotation marks
-  **3 Missing Quotations 0%**
Matches that are still very similar to source material
-  **11 Missing Citation 1%**
Matches that have quotation marks, but no in-text citation
-  **0 Cited and Quoted 0%**
Matches with in-text citation present, but no quotation marks

Top Sources

- 0%  Internet sources
- 0%  Publications
- 12%  Submitted works (Student Papers)

Integrity Flags

0 Integrity Flags for Review

Our system's algorithms look deeply at a document for any inconsistencies that would set it apart from a normal submission. If we notice something strange, we flag it for you to review.

A Flag is not necessarily an indicator of a problem. However, we'd recommend you focus your attention there for further review.

Match Groups

- 65 Not Cited or Quoted 10%**
Matches with neither in-text citation nor quotation marks
- 3 Missing Quotations 0%**
Matches that are still very similar to source material
- 11 Missing Citation 1%**
Matches that have quotation marks, but no in-text citation
- 0 Cited and Quoted 0%**
Matches with in-text citation present, but no quotation marks

Top Sources

- 0% Internet sources
- 0% Publications
- 12% Submitted works (Student Papers)

Top Sources

The sources with the highest number of matches within the submission. Overlapping sources will not be displayed.

1	Submitted works		
Islingtoncollege on 2025-05-16			4%
2	Submitted works		
Iacademy on 2025-05-14			2%
3	Submitted works		
Asia Pacific University College of Technology and Innovation (UCTI) on 2022-06-19			1%
4	Submitted works		
Southern New Hampshire University - Continuing Education on 2020-04-26			<1%
5	Submitted works		
AlHussein Technical University on 2024-01-31			<1%
6	Submitted works		
The College of West Anglia on 2024-04-18			<1%
7	Submitted works		
Chester College of Higher Education on 2015-05-07			<1%
8	Submitted works		
University of North Texas on 2023-03-24			<1%

Table of Contents

1	Introduction	1
2	Tools used	2
3	Aims and Objective	3
4	GUI wireframe	4
5	Class Diagram.....	5
5.1	Gym Member class Diagram.....	5
5.2	Regular Member	6
5.3	Premium Member.....	7
5.4	Gym GUI	8
5.5	Final class Diagram.....	9
6	Pseudocode	10
6.1	Gym Member	10
6.2	Regular Member	13
6.3	Premium Member.....	16
6.4	Gym GUI	19
7	Method description.....	44
7.1	Gym Member	44
7.2	Regular Member	45
7.3	Premium Member.....	46
7.4	Gym GUI	47
8	Testing	50
8.1	Test 1	50
8.2	Test 2	52
8.3	Test 3.....	54

8.3.1	Test case for mark Attendance	54
8.3.2	Test case for Upgrade plan	56
8.4	Test 4	59
8.4.1	Test case for calculate discount	59
8.4.2	Test case for pay due amount	60
8.4.3	Test case for revert regular member	63
8.4.4	Test case for Premium Member	65
8.5	Test case for save to file	67
8.5.1	Read from file	70
9	Error detection and correction	73
9.1	Syntax error.....	73
9.2	Run time error	75
9.3	Logical error	77
10	Conclusion.....	79
11	References	80
12	Appendix	81

Table of figures

Figure 1: Java	1
Figure 2:BLUEJ	2
Figure 3:Gym GUI	4
Figure 4: Test 1	50
Figure 5: Test 1 Output	51
Figure 6:Test 2 Output	52
Figure 7:Test 2 Output	53
Figure 8: Test 3 Output	54
Figure 9:Test 3 activate membership	55
Figure 10:Test 3 mark Attendance	55
Figure 11: test 3 adding regular member for upgrade plan	56
Figure 12:Test 3 active membership for upgrade plan	57
Figure 13:Test 3 marking attendance 38 for upgrade plan	57
Figure 14:test 3 successfully upgrade the plan	58
Figure 15: Test 4 adding premium member for discount	59
Figure 16:Test 4 output of discount amount	60
Figure 17:test case of adding premium member for pay due amount	61
Figure 18:Test case for pay due amount output	62
Figure 19: Adding regular member for regular member test case	63
Figure 20:Test case for revert regular member output	64
Figure 21: Adding premium member for test case for premium member	65
Figure 22:Test case for revert regular member	66
Figure 23:Adding regular member for test case of save to file button	67
Figure 24:Adding premium member for test case for save to file	68
Figure 25:Click save to file button	69
Figure 26:Adding regular member for read from file test	70
Figure 27:Adding premium member for test case for read from file	71
Figure 28:Output of test case of read from file	72
Figure 29:Detecting syntax error	73
Figure 30:Solving syntax error	74

Figure 31:Detection of run time error in code	75
Figure 32;Detection of run time error in GUI	75
Figure 33:Correction of run time error in code.....	76
Figure 34:Correction of run time error in GUI	76
Figure 35:Detection of logical error code.....	77
Figure 36:Correcting the logical error	78

Table of tables

Table 1Gym Member class Diagram:	5
Table 2:Regular Member class Diagram	6
Table 3:Premium Member class Diagram	7
Table 4: GymGUI class diagram	8
Table 5: Final class Diagram	9
Table 6:Test 1	50
Table 7:Test 2	52
Table 8: Test 3	54
Table 9: Test 3 upgrade plan	56
Table 10: test case for pay due amount	61
Table 11: Test case for revert regular member	63
Table 12: Test case for premium member.....	65
Table 13: Test case for save to file	67
Table 14:Test case of Read From file	70

1 Introduction

Java is a popular programming language. Many companies use Java to develop app, desktop app, games and many more. For this gym management system I have also used java to perform this task. This project was given as the coursework of 1st year student of programming. The gym Member consists of Gym Member which is the super class followed by Regular member and premium Member where gym member is the abstract class which have a abstract method name mark attendance which helps the program to keep the child class attendance differently according to their needs. The application includes a graphical user interface built with Java Swing.



Figure 1: Java

2 Tools used

Blue J is a free and open-source Integrated Development Environment (IDE) for Java. It was designed for beginners who want to learn java. It was developed by Michael Kolling and John Rosenberg at Monash University, Australia. This project started in late 1990s as a port of “Blue” pedagogical language created by Michael Kolling as part of his PhD work. The first version of Blue J 1.0 was released on August 23,1999. The project was support from Microsystems later Oracle acquired it. The latest version of Blue J is 5.4.1 which was released in Sep 19,2024.



Figure 2:BLUEJ

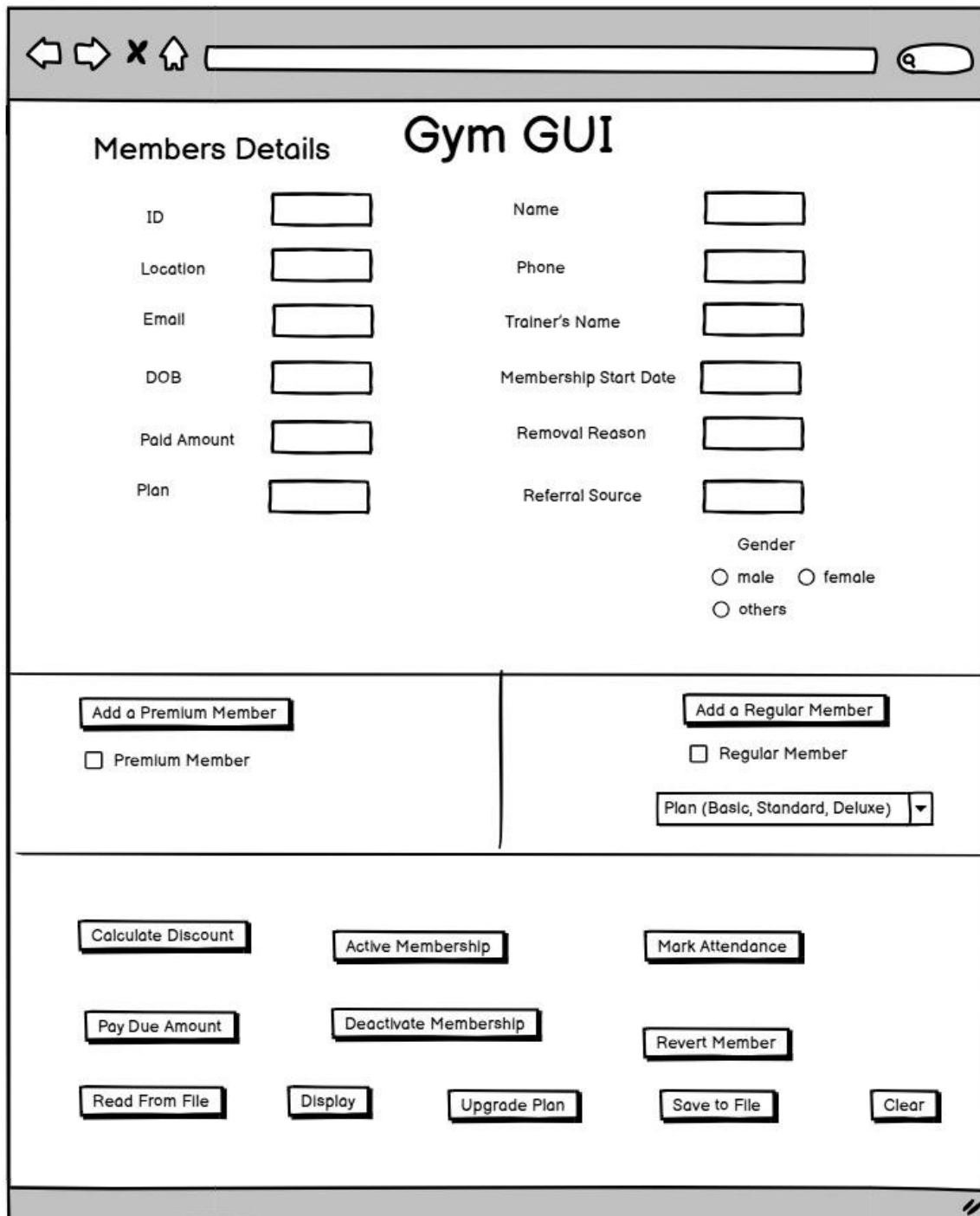
3 Aims and Objective

This report represents the gym management system given as coursework for the second semester student. The aim of the coursework is to implement the real-world scenario which consist of parent class Gym Member followed by Regular Member and Premium Member which have unique attributes and common attributes as followed by Gym Member.

The Objectives of the project are as follows: -

- To learn and implementation of GUI using swing and awt.
- To learn the concepts and implement in real-life scenario.
- To learn about button action Listener and to implement exception in real-life scenario.
- To learn to making design and layout.

4 GUI wireframe



The image shows a wireframe for a 'Gym GUI' application. It features a standard window header with navigation icons and a search bar. The main content area is divided into three sections. The top section, titled 'Members Details', contains two columns of input fields for member information. The middle section has two buttons for adding members, each with a checkbox and a plan dropdown. The bottom section is a grid of action buttons for managing members.

Gym GUI

Members Details

ID		Name	
Location		Phone	
Email		Trainer's Name	
DOB		Membership Start Date	
Paid Amount		Removal Reason	
Plan		Referral Source	

Gender
☐ male ☐ female
☐ others

☐ Premium Member

☐ Regular Member
 ▾

Figure 3:Gym GUI

5 Class Diagram

Class Diagram is a type of static structure that describes the structure of a system classes, their attributes, and method. In this project the class diagram represents the structure of Gym Management System and the relationship between their classes. This project is totally based on Object Oriented principal such as inheritance, encapsulation, abstraction, etc.

5.1 Gym Member class Diagram

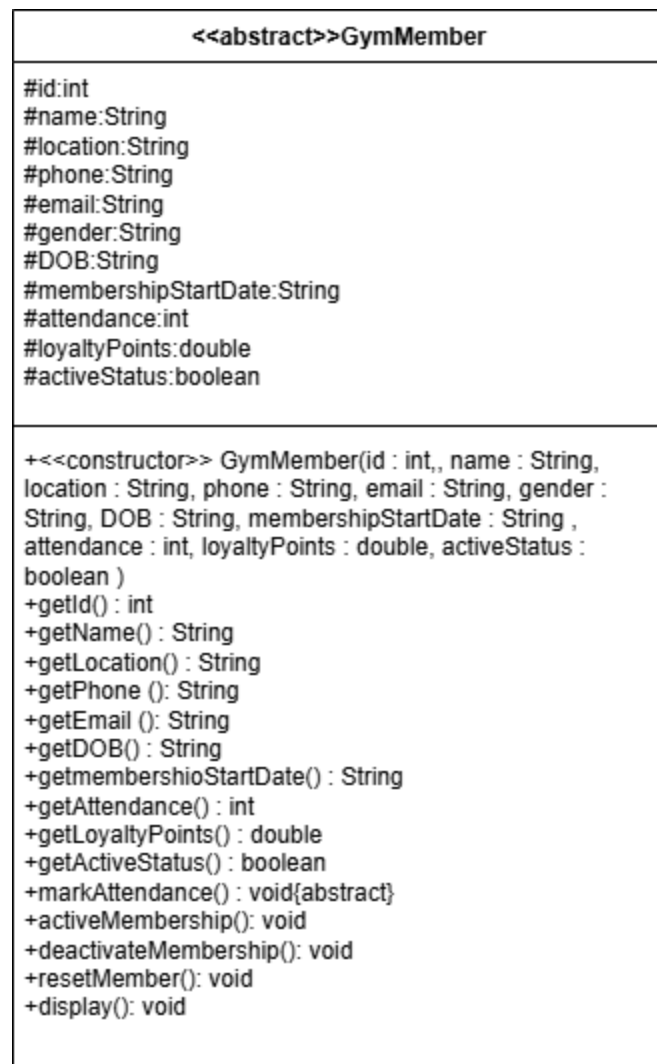


Table 1Gym Member class Diagram:

5.2 Regular Member

RegularMember
-attendanceLimit : int -isEligibleForUpgrade : boolean -removalReason : String -referralSource : String -plan : String -price : double
+<<constructor>>RegularMember(attendanceLimit : int, isEligibleForUpgrade : boolean, removalReason : String, referralSource : String, plan : String, price : double) +getAttendanceLimit() : int +getIsEligibleForUpgrade() : boolean +getRemovalReason() : String +getReferralSource() : String +getPlan() : String +getPrice() : double +markAttendance() : void +getPlanPrice() : double +upgradePlan() : String +revertRegularMember() : void +display() : void

Table 2:Regular Member class Diagram

5.3 Premium Member

PremiumMember
<pre> -premiumCharge : double -personalTrainer : String -isFullPayement :boolean -paidAmount : double -discountAmount : double </pre>
<pre> +<<constructor>>Premium Member (premiumCharge : boolean, personalTrainer : String, isFullPayement : boolean, paidAmount : double, discountAmount : double) +getPremiumCharge() : double +getPersonalTrainer() : String +getPaidAmount() : boolean +getPaidAmount() : double +getDiscountAmount() : double +markAttendance () : void +payDueAmount() : String +calculateDiscount() : void +revertPremiumMember() : void +display(): void </pre>

Table 3:Premium Member class Diagram

5.4 Gym GUI

GymGUI
-frame : JFrame -idField, nameField, locationField, phoneField, emailField, referralField, paidField removalField, trainerField, memberField, reguPriceField, preCharField ,disAmmountField : JTextField -dobYearBox, dobMonthBox, dobDayBox: JComboBox<String> -planBox: JComboBox<String> -genderGroup : ButtonGroup -members : ArrayList<GymMember> -regularMemBtn, premiumbtn, activateMembershipButton, deactivateMembershipButton, markAttendancebtn, revertRegularMemberButton, revertPremiumBtn, upgradeButton, discountButton;
+GymGUI() +createGUI () : void -getYears () : String [] -getMonths () : String [] -getDays() : String [] -isValidId(int): GymMember +addRegularMember () : void +addPremiumMember () : void +addactivateMembership() : void +addeactivateMembershipButton() : void +addMarkAttendancebtn(): void +addrevertRegularMemberbtn() : void +addrevertPremiumMemberbtn() : void +displayMembers() : void -clearFields() : void +calculateDiscount() : void +addUpgradePlan() : void +addPayDueAmoun() : void -saveToFile() : void +readFromFile() : void +main(String): void

Table 4: GymGUI class diagram

5.5 Final class Diagram

(Sovereign Corporate Tower, 2025)

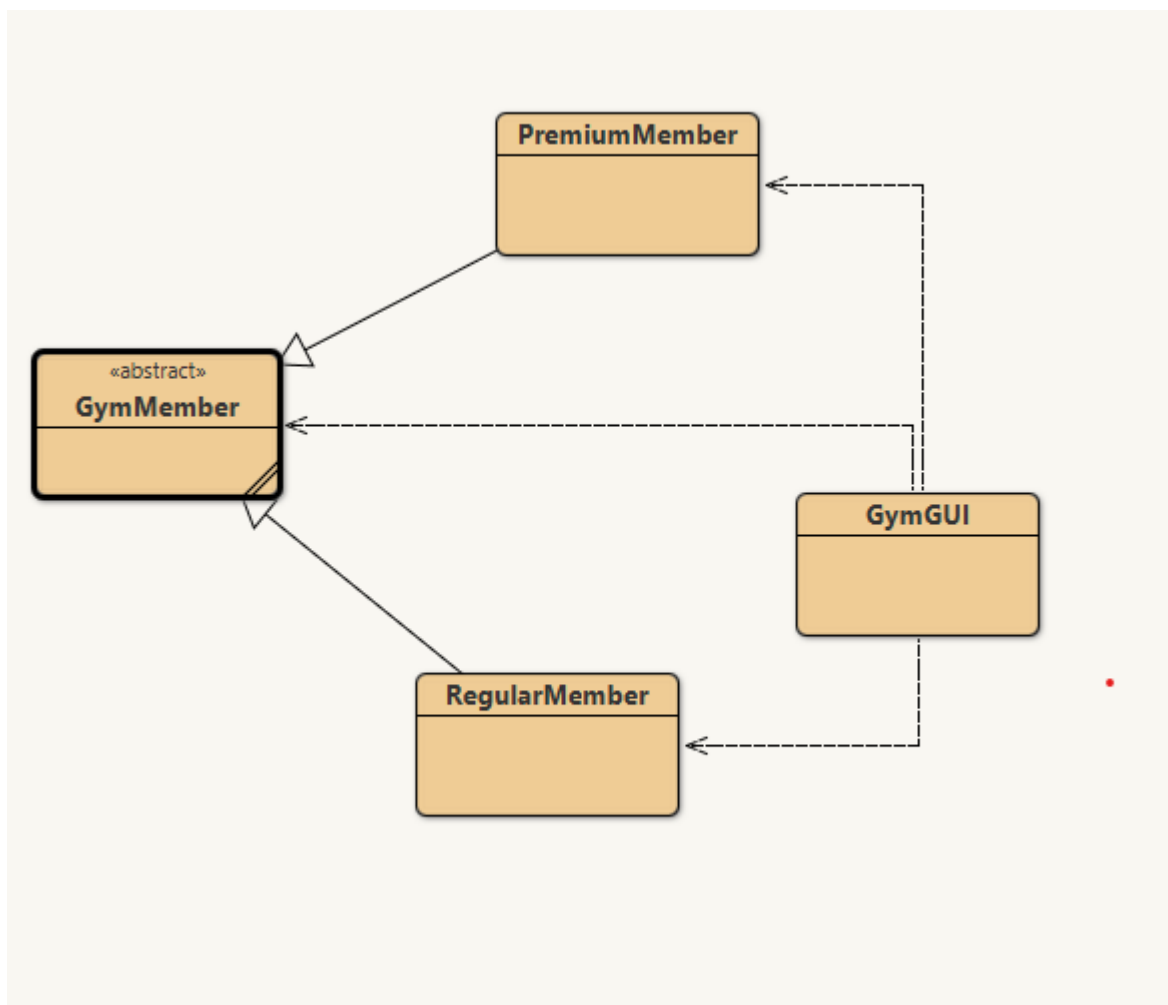


Table 5: Final class Diagram

6 Pseudocode

Pseudocode is half English and half code of writing a program to express the program logic. For my project I have created whole program to explain and express the program logic.

6.1 Gym Member

CREATE a parent class which is also a abstract class GymMember

DECLARE instance variable id as int with protected access

DECLARE instance variable name as String with protected access

DECLARE instance variable location as String with protected access

DECLARE instance variable phone as String with protected access

DECLARE instance variable email as String with protected access

DECLARE instance variable gender as String with protected access

DECLARE instance variable DOB as String with protected access

DECLARE instance variable memberShipStartDate as String with protected access

DECLARE instance variable attendance as int with protected access

DECLARE instance variable loyaltyPoints as double with protected access

DECLARE instance variable activeStatus as Boolean with protected access

CONSTRUCTOR GymMember(id, name, location, phone, email, gender, DOB, membershipStartDate, attendance, loyaltyPoints, activeStatus)

SET this.id = id

SET this.name= name

SET this.location = location

```
SET this.phone= phone

SET this.email= email

SET this.gender= gender

SET this.DOB = DOB

SET this.membershipStartDate = membershipStartDate

SET this.attendance = attendance

SET this.loyaltyPoints= loyaltyPoints

SET this.activeStatus= activeStatus

END CONSTRUCTOR

ABSTRACT FUNCTION markAttendance()

END FUNCTION

FUNCTION activateMembership()

    SET activeStatus = true

END FUNCTION

FUNCTION deactivateMembership()

    IF activeStatus IS true THEN

        SET activeStatus = false

    END FUNCTION

FUNCTION resetMember()

    SET activeStatus = false

    SET attendance = 0

    SET loyaltyPoints = 0

END FUNCTION
```

FUNCTION display()

 OUTPUT "Member ID" + id

 OUTPUT "Name" + name

 OUTPUT "location" + location

 OUTPUT "phone" + phone

 OUTPUT "email" + email

 OUTPUT "gender" + gender

 OUTPUT "DOB" + DOB

 OUTPUT "membershipStartDate" + memberShipStartDate

 OUTPUT "attendance" + attendance

 OUTPUT "loyaltyPoints" + loyaltyPoints

 OUTPUT "activeStatus" + activeStatus

END FUNCTION

END CLASS

6.2 Regular Member

CREATE a child class as RegularMember

DECLARE instance variable as attendanceLimit as int with private access modifier as it has final static value

DECLARE instance variable isEligibleForUpgrade as Boolean with private access modifier

DECLARE instance variable removalReason as String with private access modifier

DECLARE instance variable referralSource as String with private access modifier

DECLARE instance variable plan as String with private access modifier

DECLARE instance variable price as double with private access modifier

CONSTRUCTOR RegularMember(id,name,location,phone,email,gender,DOB, membershipStartDate,referralSource)

CALL SUPER CONSTRUCTOR (id,name,location,phone,email,gender,DOB, membershipStartDate)

SET this.attendanceLimit = attendanceLimit

SET this.isEligibleForUpgrade = isEligibleForUpgrade

SET this.removalReason = removalReason

SET this.referralSource = referralSource

SET this.plan = plan

SET this.price = price

END CONSTRUCTOR

```
FUNCTION markAttendance()  
    INCREMENT attendance by 1  
    INCREMENT loyaltyPoints by 5  
    IF attence is greater or equal then attendanceLimit  
        SET isEligibleForUpgrade = true  
    END FUNCTION
```

```
FUNCTION getPlanPrice(String Plan)  
    SWITCH plan to lowercase  
    Case=basic  
        RETURN 6500  
    Case =Standard  
        RETURN 12500  
    Case Deluxe  
        RETURN 18500  
    DEFAULT  
        RETURN -1
```

```
FUNCTION upgradePlan (String plan)  
    IF not isEligibleForUpgrade THEN  
        RETURN "Not eligible for upgrade. Need more attendance"  
    END IF
```

IF new.plan to lowercase() equals plan then

RETURN "Already subscribed to this plan"

SET this.newprice= getPlanPrice

IF new price == -1 Then

RETURN "Invalid Plan Selected"

SET this.Plan = newPlan to lowercase

SET this.Price = new Price

RETURN "Plan upgrade successfully to" + newPlan

FUNCTION revertRegularMember(String removalReason)

CALL SUPER resetMember() method

SET this.isEligibleForUpgrade = false

SET this.plan = basic

SET this.price = 6500

SET this.removalReason = removalReason

END FUNCTION

FUNCTION display()

CALL SUPER display() method

OUTPUT "plan"+ plan

OUTPUT "Price"+ price

IF not removal reason is empty then

OUTPUT "removal reason" + removal reason

6.3 Premium Member

CREATE a child class name as Premium Member

DECLARE instance variable premiumCharge as double which has final value = 50000 with private access modifier

DECLARE instance variable personalTrainer as String with private access modifier

DECLARE instance variable isFullPayement as Boolean with private access modifier

DECLARE instance variable paidAmount as double with private access modifier

DECLARE instance variable discountAmount as double with private access modifier

CREATE CONSTRUCTOR PremiumMember(id, name, location, phone, email, gender ,
DOB, memberShipStartDate, personalTrainer)

CALLSUPERCONSTRUCTOR(id,name,location,phone,email,gender,DOB,membership
StartDate)

SET this.premiumCharge = premiumCharge

SET this.personalTrainer = personalTrainer

SET this.isFullPayemet = isFullPayement

SET this.paidAmount = paidAmount

SET this.discountAmount = discountAmount

END CONSTRUCTOR

FUNCTION payDueAmount()

IF this.isFullPayement is true then

RETURN "Payement is already complete"

```
IF this.paidAmount + amount > premiumCharge then
RETURN "payment amount exceeds the premium charge"
INCREMENT paidAmount by amount
IF paidAmount == premiumCharge THEN
SET isFullPayment = true
SET remainigAmount = PremiumCharge – paidAmount
RETURN "Payment Successful. Remaining amount" + Remaining amount
END FUNCTION
```

```
FUNCTION calculateDiscount()
IF isFullPayment is true THEN
SET discountamount = premiumCharge – paidAmount
OUTPUT "Discount calculated" + discountAmount
END IF
OUTPUT "No discount available. Complete Payment first "
END FUNCTION
```

```
FUNCTION revertPremiumMember()
CALL SUPER resetMember() method
SET personalTrainer = ""
SET isFullPayment = false
SET paidAmount =0
SET discountAmount = 0
```

END FUNCTION

FUNCTION display()

CALL SUPER display() method

DISPLAY "Personal Trainer " + personalTrainer

DISPLAY "Paid Amount" + paidAmount

DISPLAY " Payment Status " +isFullPayement

SET remainingAmount = premiumCharge – paidAmount

DISPLAY "Remaining amount" + remainingAmount

IF isFullPayment is true THEN

DISPLAY "Discount Amount " + discountAmount

END FUNCTION

END CLASS

6.4 Gym GUI

Create a child class as GymGUI

DECLARE frame as JFrame

DECLARE idField, nameField, locationField, phoneField, emailField, referralField, paidField, removalField, trainerField reguPriceField, preCharField, diaAmountField as JTextField

DECLARE dobYearBox, dobMonthBox, dobDayBox as JComboBox<String>

DECLARE msYearBox, msMonthBox, msDayBox as JComboBox<String>

DECLARE genderGroup as ButtonGroup

DECLARE malebutton, femalebutton, otherbutton as JRadioButton

DECLARE members as ArrayList<GymMember>

DECLARE regularmembbtn, premiumbtn, activateMembershipBtn, deactivateMembershipBtn, markAttendancebtn, revertRegularMemberBtn, revertPremiumBtn, upgradeButton, discountButton as JButton

FUNCTION GymGUI()

 CALL createGUI()

END FUNCTION

FUNCTION createGUI()

 CREATE JFrame frame ("Gym Management System")

 SET frame defaultCloseOperation to JFrame.EXIT_ON_CLOSE

 SET layout of frame to null

 SET size of the layout (width 900, height 900)

SET background color to rgba(240,240,240)

SET frame contentPane to the same as backgroundColor

CREATE JPanel headerPanel

SET layout of headerPanel to null

SET headerPanel bounds to (x=0,y=0,width=900,height=50)

SET headerPanel color to rgb(42,68,80)

ADD headerPanel to frame

CREATE JLabel titleLabel ("Gym Management System")

SET bounds to titleLabel(x=300,y=20,width=900,height=50)

SET foreground color to titleLabel to Color.WHITE

SET titleLabel font and size to ("serif",Font.BOLD,14)

SET a light gray line border around the main panel

ADD titleLabel to headerPanel

CREATE a new JPanel name as mainPanel

SET layout of main panel to null

SET bounds of mainPanel to x:30,y:100,width:820,height:410

SET background color of mainPanel to white

SET border of mainPanel to a line border with color (200,200,200)

ADD mainPanel to the frame

CREATE a new JLabel named meminfo with text "member information"

SET bounds of membinf to x:10,y:10,width:380,height:25

SET font of membinf to “Segoe UI”, style bold, size 16

ADD membinf to mainPanel

CREATE a new JLabel named as idLabel with text “Id”

SET bounds of idLabel to x:20,y:50,width:150,height=25

Add idLabel to mainPanel

CREATE a new JTextField names as idField

SET bounds to idField to x:180,y:50,width:200,height=30

ADD idField to mainPanel

CREATE a new JLabel named as nameLabel with text “name”

SET bounds to nameLabel to x:20,y:90,width:150,height:30

ADD nameField to mainPanel

CREATE a new JTextField named as nameField

SET bounds to nameField to x:180,y:50,width:200,height:30

ADD nameField to mainPanel

CREATE a new JLabel name as locationLabel with text “location”

SET bounds to locationLabel to x:20,y:130,width:150,height:30

ADD locationLabel to mainPanel

CREATE a new JTextField named locationField

SET bounds to locationField to x:180,y:130,width:200,height:30

ADD locationField to mainPanel

CREATE a new JLabel named as phoneLabel with text "Phone"

SET bounds to phoneLabel to x:20,y:170,width:200,height:30

ADD phoneLabel to mainPanel

CREATE a new JTextField named as phoneField

SET bounds to phoneField to x:180, y:170, width:150, height:30

ADD phoneField to mainPanel

CREATE a new JLabel named emailLabel with text "email"

SET bounds to emailLabel to x:20,y:210,width:150,height:30

ADD emailLabel to mainPanel

CREATE a new JTextField named emailField

SET bounds to emailField to x:180,y:210,width:200,height:30

ADD emailField to mainPanel

CREATE a new JLabel named as genderlabel with text "gender"

SET bounds to genderLabel to x:20,y:250,width:150,height:30

ADD genderLabel to mainPanel

CREATE a new JRadioButton name as malebutton with text “male”

SET bounds to malebutton to x:180,y:250,width:100,height:30

SET background color of malebutton to white

ADD male button to mainPanel

CREATE a new JRadioButton named as femalebutton with text “female”

SET bounds to femalebutton to x:250,y:250,width:100,height:30

SET background color of femalebutton to white

ADD femalebutton to mainPanel

CREATE a new JRadiobutton named as otherbutton with text “other”

SET bounds to otherbutton to x:320,y:250,width:100,height:30

SET background color of otherbutton to white

ADD otherButton to mainPanel

CREATE a new ButtonGroup named as genderGroup

ADD malebutton to genderGroup

ADD femalebutton to genderGroup

ADD otherbutton to genderGroup

CREATE a new JLabel named as dobLabel with the text “DOB”

SET bounds to dobLabel to x:20,y:250,width:140,height:30

ADD dobLabel to mainPanel

CREATE a JComboBox named as dobYearBox with items from getYears()

SET bounds to dobYearBox to x:20,y:290,width:150,height:30

ADD dobYearBox to mainPanel

CREATE a JComboBox named as dobMonthBox with items from getMonth()

SET bounds to dobMonthBox to x: 250,y:285,width:60,height:25

ADD dobMonthBox to mainPanel

CREATE a JComboBox named as dobDayBox with items from getDay()

SET bounds to dobDayBox to x:315,y:285,width:60,height:25

ADD dobDayBox to mainPanel

CREATE a new JLabel named as membershipLabel with text "Membership Details"

SET bounds to membershipLabel to x:440,y:10,width:380,height:25

SET font of membershipLabel to "segoe UI",style Bold, size:16

ADD membershipLabel to mainPanel

CREATE a new JLabel named as membershipStart with text "Membership Start"

SET bounds to membershipStart to x:440,y:50,width:150,height:30

ADD membershipStart to mainPanel

CREATE a JComboBox named as msYearBox

SET bounds to msYearBox to x:600,y:50,width:60,height:25

ADD msYearBox to mainPanel

CREATE a JComboBox named as msMonthBox

SET bounds to msMonthBox to x:670,y:50,width:60,height:25

ADD msMonthBox to mainPanel

CREATE a JComboBox named as msDayBox

SET bounds to msDayBox to x:750,y:50,width:60,height:25

ADD msDayBox to mainPanel

CREATE a JLabel named as referralLabel with text "Referral Source"

SET bounds to referralLabel to x:440,y:100,width:160,height:30

ADD referralLabel to mainPanel

CREATE a new JTextField named as referralField

SET bounds to referralField to x:600,y:100,width:200,height:30

ADD referralField to mainPanel

CREATE a new JLabel named as paidLabel with text "Paid Amount"

SET bounds to paidLabel to x :440,y:140,width:150,height:30

ADD paidLabel to mainPanel

CREATE a new JTextField named as paidField

SET bounds to paidField to x:600,y:140,width:200,height:30

ADD paidField to mainLabel

CREATE a new JLabel named as removLabel text as "Removal Reason"

SET bounds to removLabel to x:440,y:180,width:150,height:30

ADD removLabel to mainPanel

CREATE a new JTextField named as removalField

SET bounds to removalField to x:600,y:180,width:200,height:30

ADD removalField to mainPanel

CREATE a new JLabel named as trainerLabel with text "Trainer Name"

SET bounds to trainerLabel to x:440,y:220,width:150,height:30

ADD trainerLabel to mainPanel

CREATE a new JTextField named as trainerField

SET bounds to trainerField to x:600,y:220,width:200,height:30

ADD trainerField to mainPanel

CREATE a new JLabel named as planLabel with text “Membership Plan”

SET bounds to planLabel to x:440,260,150,30

ADD planLabel to mainPanel

CREATE a String array named plan as values “basic” “standard” “deluxe”

CREATE a JComboBox named as planBox

SET bounds to planBox to x:600,y:250,width:200,height:25

SET background color of planBox to white

ADD planBox to mainPanel

CREATE a JLabel named discountLabel with text “Discount Amount”

SET bounds to discountLabel to x:440,y:300,width:150,height:30

ADD discountLabel to mainPanel

CREATE a JTextField named disAmounField

SET bounds to disAmounField to x:600,y:300,width:200,height:30

ADD disAmounField to mainPanel

CREATE a new JLabel named regularPlanPrice with text “Regular Plan Price”

SET bounds to regularPlanPrice to x:440,y:340,width:150,height:30

ADD regularPlanPrice to mainPanel

CREATE a new JTextField named reguPriceField

SET bounds to reguPriceField to x:600,y:340,width:200,height:30

SET text of reguPriceField to "6500"

SET editable changes of reguPriceField be false

ADD reguPriceField to mainPanel

CREATE a new JLabel named premiumPlanPrice with text "Premium Plan Price"

SET bounds to premiumPlanPrice to x:440,y:380,width:150,height:30

ADD premiumPlanPrice to mainPanel

CREATE a new JTextField named preCharField

SET bounds to preCharField to x:600,y:380,width:200,height:30

SET text to preCharField as "50000"

SET editable text to false

ADD preCharField to mainPanel

CREATE a new JPanel named as buttonPanel

SET layout of buttonPanel to null

SET bounds to buttonPanel to x:30,y:500,width:820,height:400

ADD buttonPanel to frame

CREATE a JButton named as regularmemBtn with text " Add Regular Member"

SET bounds to regularmemBtn to x:30,y:30,width:200,height:30

SET background color to headerColor

SET font of regularmemBtn to "Poppins", style PLAIN, size 14

ADD regularmemBtn to buttonPanel

CREATE a JButton named as premiumBtn with text "Add Premium Member"

SET bounds to premiumBtn to x:300,y:30,width:200,height:30

SET font of premiumBtn to "Poppins", style PLAIN, size 14

ADD premiumBtn to buttonPanel

WHEN regularmemBtn is clicked:

CALL addRegularMember() method

WHEN premiumBtn is clicked:

CALL addPremiumMember() method

CREATE a new JButton named activateMembershipBtn with text "Activate Membership"

SET bounds to activateMembershipBtn to x:600,y:30,width:200,height:30

SET font of activateMembershipBtn to "Poppins", Style.PLAIN, 14

ADD activateMembershipBtn to buttonPanel

WHEN activateMembershipBtn is clicked:

CALL activateMembershipBtn() method

CREATE a new JButton named as deactivateMembershipButton with text “ Deactivate Membership”

SET bounds to deactivateMembershipButton to x:30,y:90,width:200,height:30

SET font to deactivateMembershipButton to “Poppins”,Style.PLAIN,14

ADD deactivateMembershipButton to buttonPanel

WHEN deactivateMembershipButton is clicked:

CALL deactivateMembershipButton() method

CREATE a JButton named markAttendancebtn with text “Mark Attendance”

SET bounds to markAttendancebtn to x:300,y:90,width:200,height:30

SET font to markAttendance to “Poppins”,Style.PLAIN,14

ADD markAttendancebtn to buttonPanel

WHEN markAttendancebtn is clicked:

CALL addmarkAttendancebtn() method

CREATE a JButton named revertRegularMemberbtn with text “Revert Regular Member”

SET bounds to revertRegularMemberbtn to x:600,y:90,width:200,height:30

SET font to revertRegularMemberbtn to “Poppins”,Style.PLAIN,14

ADD revertRegularMemberbtn to buttonPanel

WHEN revertRegularMemberbtn is clicked:

CALL `addrevertRegularMemberbtn()` method

CREATE a JButton named `revertPremiumMember` with text “Revert Premium Member”

SET bounds to `revertPremiumMember` to `x:30,y:150,width:200,height:30`

SET font to `revertPremiumMemberbtn` to “Poppins”,`Style.PLAIN,14`

ADD `revertPremiumMemberbtn` to `buttonPanel`

WHEN `revertPremiumMemberbtn` is clicked:

CALL `addrevertPremiumMemberbtn()` method

CREATE a new JButton named `displayMemberButton` with text “Display Members”

SET bounds to `displayMemberButton` to `x:300,y:150,width:200,height:30`

SET font to `displayMemberButton` to “Poppins”,`Style.PLAIN,14`

ADD `displayMemberButton` to `buttonPanel`

WHEN `displayMemberButton` is clicked:

CALL `displayMembers()` method.

CREATE a new JButton named `clearFieldsButton` with text “Clear Fields”

SET bounds to `clearFieldsButton` to `x:600,y:150,width:200,height:30`

SET font to `clearFieldsButton` to “Poppins”,`Style.PLAIN,14`

ADD `clearFieldsButton` to `buttonPanel`

WHEN `clearFieldsButton` is clicked:

CALL clearFields() method

CREATE a new JButton named discountButton with text "Calculate Discount"

SET bounds to discountButton to x:30,y:210,width:200,height:30

SET font to discountButton to "Poppins",Style.PLAIN,14

ADD discountButton to buttonPanel

WHEN discountButton is clicked:

CALL calculateDiscount() method

CREATE a new JButton named upgradeButton with text "Upgrade Plan"

SET bounds to upgradeButton to x:300,y:210,width:200,height:30

SET font to upgradeButton to "Poppins",Style.PLAIN,14

ADD upgradeButton to buttonPanel

WHEN upgradeButton is clicked:

CALL addUpgradePlan() method

CREATE a new JButton named payDueAmount with text "Pay Due Amount"

SET bounds to payDueAmount to x:600,y:210,width:200,height:30

SET font to payDueAmount to "Poppins",Style.PLAIN,14

ADD payDueAmount to buttonPanel

WHEN payDueAmount is clicked:

CALL addPayDueAmount() method

CREATE a new JButton named savetoFileButtpn with text "Save to file"

SET bounds to savetoFileButton to x:30,y:270,width:200,height:30

SET font of savetoFileButton to "Poppins",Style.PLAIN,14

ADD savetoFileButton to buttonPanel

WHEN savetoFileButton is clicked:

CALL savetofile() method

CREATE a new JButton named readFromFilebtn with text "Read from file"

SET bounds to readFromFileBtn to x:300,y:270,width:200,height:30

SET font of readFromFileBtn to "Poppins",Style.PLAIN,14

ADD readFromFileBtn to buttonPanel

WHEN readFromFileBtn is clicked:

CALL readFromFile() method

SET FRAME visible to true

METHOD getYears():

CREATE a String array named years with size 50

FOR I from 0 to 49:

Set years[i] to (2025 - i) converted to String

RETURN to years array

METHOD getMonths():

 CREATE a String array named years with size 12

 FOR I from 1 to 12:

 Set months[i] to (i+1) converted to String

 RETURN to months array

METHOD getDays():

 CREATE a String array named years with size 7

 FOR I from 1 to 7:

 Set years[i] to (i+1) converted to String

 RETURN to months array

METHOD addRegularMember()

TRY:

 Get ID from idField and convert to integer

 IF id already exist:

 SHOW error message

 GET name, location, phone, email, and referral form text field

 IF maleButton is selected:

gender = "male"

ELSE IF femaleButton is selected:

gender="female"

ELSE IF otherButton is selected:

gender = "other"

ELSE

SHOW ERROR MESSAGE

IF any of the DOB or MembershipStart combo box are not selected

SHOW ERROR MESSAGE

EXIT the method

CONSTRUCT DOB String as "year-month-day" from DOB ComboBox

CONSTRUCT membershipStartDate String as "year-month-day" from membership Combo box

IF any required text field are empty:

SHOW error message

IF phone is NOT exactly 10 digits:

SHOW Error Message

IF email is NOT in invalid format

SHOW Error Message

CREATE and add the Regular Member

CREATE RegularMember Object with provided details

SHOW success message

CATCH NumberFormatException:

SHOW error message

METHOD addPremiumMember()

TRY:

Get ID from idField and convert to integer

IF id already exist:

SHOW error message

GET name, location, phone, email, and referral form text field

IF maleButton is selected:

gender = "male"

ELSE IF femaleButton is selected:

gender="female"

ELSE IF otherButton is selected:

gender = "other"

ELSE

SHOW ERROR MESSAGE

IF any of the DOB or MembershipStart combo box are not selected

SHOW ERROR MESSAGE

EXIT the method

CONSTRUCT DOB String as "year-month-day" from DOB ComboBox

CONSTRUCT membershipStartDate String as "year-month-day" from membership Combo box

IF any required text field are empty:

SHOW error message

IF phone is NOT exactly 10 digits:

SHOW Error Message

IF email is NOT in invalid format

SHOW Error Message

CREATE and add the PremiumMember

CREATE PremiumMember Object with provided details

SHOW success message

CATCH NumberFormatException:

SHOW error message

METHOD addactivateMembership()

TRY:

Get ID from idField and convert to integer

CALL isValidID and store the result in a variable name member

IF member == null:

SHOW error message

EXIT the method

Call activateMembership() on the member

SHOW success message

CLEAR all input fields

CATCH NumberFormatException:

SHOW error message

METHOD adddctivateMembership()

TRY:

Get ID from idField and convert to integer

CALL isValidID and store the result in a variable name member

IF member == null:

SHOW error message

EXIT the method

Call deactivateMembership() on the member

SHOW success message

CLEAR all input fields

CATCH NumberFormatException:

SHOW error message

METHOD addmarkAttendanceBtn()

TRY:

GET ID from idField and convert to integer

CALL isValidID and store the result in a variable name member

IF member is not active

SHOW Error Message

CALL markattendance() on the member

SHOW success message

CATCH NumberFormatException:

SHOW error message

METHOD addRevertRevertRegularMemberbtn()

TRY:

 GET ID from idField and convert to integer

 CALL isValidID and store the result in a variable name member

IF member is not active

SHOW Error Message

IF member is NOT instance of Regular Member

SHOW error message

EXIT the Method

GET text from removalField and store in a reason

IF reason is empty

SHOW error message

CALL revertRegularMember on RegularMember

SHOW success message

CATCH NumberFormatException:

 SHOW error message

MEHOD addrevertRevertPremiumMemberbtn()

TRY:

 GET ID from idField and convert to integer

 CALL isValidID and store the result in a variable name member

IF member is not active

 SHOW Error Message

IF member is NOT instance of Premium Member

 SHOW error message

EXIT the Method

GET text from removalField and store in a reason

IF reason is empty

 SHOW error message

CALL revertPremiumMember on PremiumMember

SHOW success message

CATCH NumberFormatException:

 SHOW error message

METHOD Display Members()

CREATE a new JFrame named display with text "Member List"

SET defaultCloseOperation to DISPOSE on exit

SET size to width:600, height:600

SET layout to null

CREATE a new JTextArea named displayArea

SET editable of displayArea to false

CREATE a new JScrollPane named scrollPane

SET scrollPane bounds to x: 10,y:10,width:560,height:540

ADD scrollPane to displayPane

IF member is empty:

SET text to "No members found"

CREATE a stringBuilder named as displayText

For each member in the memberlist

IF member is regularMember

APPEND regularMember

IF member is PremiumMember

APPEND premiumMember

APPEND members details like ID, name, location, Phone, Email, Gender, DOB, Start Date, Attendance, Loyalty Points, Active Status

IF member is instance of regularMember:

APPEND plan and price

IF removalReason is not null and removalReason is not empty

APPEND removal Reason

ELSE IF member is instance of premiumMember:

APPEND PersonalTrainer, PaidAmount and Payement Status

IF remainingAmount= preMember * getPremiumCharge

APPEND remainingAmount

IF premium member get fullPayement

APPEND discountAmount

SET displayFrame visible to true

METHOD savetoFile()

TRY:

OPEN a filewriter to wrie to "membersDetails.txt"

WRITE header Line with Membership details

Write colums header like ID, Name,location, phone, email, Membership Start date , Plan, Price, Attendance, loyalty Points, Active Status, Full Payment, Discount amount, Paid Amount, Net Amount Paid

IF member is Regular Member:

Write formatted details like ID, Name, location, phone, email, Membership Start date,plan, price, Attendance, loyalty Points, Active status, full payment, discount amount, paid amount, net amount paid

ELSE IF member is Premium Member

Write formatted details in row in order Like ID, Name, location, phone, email, Membership Start date, plan, price, Attendance, loyalty points, Active status, full payment , discount amount, paid Amount, net amount paid

SHOW success message

CATCH NumberFormatException

SHOW error message

METHOD readFromFile()

CREATE a JTextArea and set editable to false

SET size to rows 20, columns 80

ADD text area to scrollPane

TRY:

OPEN the file "Membership Details" using a Scanner

WHILE there are more lines in the file

 READ each line and append it to text area

SHOW the scrollable text area in a message dialog titled "Member details"

CATCH IOException

SHOW error message

(Sovereign Corporate Tower, 2025)

7 Method description

7.1 Gym Member

- `getID()`: This is a integer return type method that returns an integer value of the id of the member.
- `getName()`: This is a String return type method that returns an String value of the full name of the member.
- `getLocation()`: This is String return type method that returns the location of the member.
- `getPhone()`: This is a String return type method that return the phone number of the member.
- `getGender()`: This is a String return type method that return the gender of the member.
- `getDOB()`: This a String return type method that return the DOB of the member.
- `getMembershipStartDate`: This is a String type method that returns the date member started their membership.
- `getAttendance()`: This is a integer return type that return the attendance of the member.
- `getLoyaltyPoints()`: This is a double return type that returns the loyalty points of the member.
- `getActiveStatus()`: This is a Boolean type method that returns the current active status true or false.
- `markAttendance()`: This is an abstract method that must be implement in the base class of the Gym Member.
- `activateMembership()`: This method set the active status of Gym Member to true.
- `deactivateMembership()`: This method set the active status of Gym Member to false.
- `resetMember()`: This method set the active status to true and set the attendance and loyalty point to 0
- `displayMember()`: This method prints the details of Gym Member in the console.

7.2 Regular Member

- `getAttendanceLimit()`: This is a integer type method return maximum number of days the regular member is allowed to attend the gym.
- `isEligibleForUpgrade()`; This is Boolean type method return the member qualifies for upgrade of Regular Member.
- `getReferralSource()`: This is String type method return the reason provided to remove the member from the system.
- `getPlan()`: This is a String type method return the member how they referred to the gym.
- `getPrice()`: This is a double type method that return the price for the regular member.
- `markAttendance()`: This method updates a regular member attendance and loyalty member of the premium member.
- `getPlanPrice()`: This method returns price of the membership plan based on the plan name provided.
- `upgradePlan()`: This method attempts to upgrade a regular member current membership plan to new one.
- `revertRegularMember()`: This method is used to reset a regular member status when they are removed.
- `Display()`: This method prints all the details of premium member in the console.

7.3 Premium Member

- `getPremiumCharge()` : Returns the fixed membership amount which is 50000.
- `getPersonalTrainer()`: Returns the name of the personal trainer for the premium Member.
- `getIsFullPayment()`: Returns a Boolean value whether the member made his full payment or not.
- `getPaidAmount()`:Returns the total amount of the premium member has paid.
- `discountAmount()`: Returns the discount amount which is applied for premium member.
- `markAttendance()`:This method updates a Premium member attendance and loyalty member of the premium member.
- `payDueAmount()`: This method allows a premium member to make a payment towards their premium charge by adding the specified amount of the total paid.Lastly return a message indicating the success of payment along with remaining balance.
- `revertPremiumMember()`:This method reset the premium member by calling the parent `resetMember` method, remove the personal trainer information, set full payment to false and set discount amount to 0.
- `Display()`: This method call the parent class `display` method, and print all the details of premium member.

7.4 Gym GUI

- `getYears()`: This method creates an array of 50 years starting from 2025 and returns the current year.
- `getMonths()`: This method creates an array of months as String from 1 to 12 and returns the current month.
- `getDays()`: This method creates an array of 30 days as String and returns the current day .
- `addRegularMember()`: This method handles the addition of a new regular gym member. Firstly It check the proper validation of the id and id should be unique and must verify required fields such as name, location, phone number, email and referral source and the field could not be empty. The method ensured the phone number should be exactly with 10 digits and ensures the proper validation of the email. If all the validation is check it create a new regular member object when the user input all the data and adds to the member list.
- `addPremiumMember()`: This method handles the addition of a new regular gym member. Firstly It check the proper validation of the id and id should be unique and must verify required fields such as name, location, phone number, email and referral source and the field could not be empty. The method ensured the phone number should be exactly with 10 digits and ensures the proper validation of the email. If all the validation is check it create a new Premium member object when the user input all the data and adds to the member list.
- `activateMembership()`: This methods actives the membership status of the member. Firstly it parse the ID input and check if the ID is already exists using `invalidID` method. If the member is found that the membership set to active by calling the `activatemembership` method and display a success method. If the id is nt found then it give the error message.
- `adddeactivateMembership()`: This methods actives the membership status of the member. Firstly it parse the ID input and check if the ID is already exists using `invalidID` method. If the member is found that the membership set to active by calling the `deactiveMembership` method and display a success method. If the id is nt found then it give the error message.

- `addMarkAttendancebtn()`: This method is used for the mark attendance of the gym member. This method first validated the input ID and check whether the id is valid or not using `isValidId` method. If the member is found and their membership is active then it call the `markAttendance()` method to keep record for the attendance and calculate loyalty points.
- `addRevertRegularmemberbtn()`: This method reverts a Regular member to the initial state. This method first validates the input ID and check whether the id is valid or not using `isValidID` method. It check it is of Regular Member and verifies the removal reason has entered or not. If all condition are meet then it call the `revertRegulatMember` method with a given reason to reset the member attributes.
- `addRevertPremiumMember()`: This method reverts a Regular member to the initial state. This method first validates the input ID and check whether the id is valid or not using `isValidID` method. It check it is of Premium Member and verifies the removal reason has entered or not. If all condition are meet then it call the `revertPremiumMember` method with a given reason to reset the member attributes.
- `displayMembers()`: This methods create a new frame to display details of gym member. It uses a `JTextArea` inside a `JScrollPane`. The method first validates the input of ID and it checks it is valid id or not using `isValidID` method. If the member list is empty it show a message "No members found". If the member is instance of premium member and regular member it display the members details like Id, name, contact details, membership status ,plan and payment details.
- `clearFields()`: The method reset all the input fields to their default state. It clear all the text from the field like in `idField`, `nameField`, `locationField`, `emailField`, `referralField`, `paymentField` and `trainerField`. It also reset the DOB and membership Start date of `JCombbobox` and also clear the selected gender of `JRadioButton`.
- `calculateDiscount()`: This method handles the discount amount of the premium member. It first handles the input ID and check it is valid id or not. Check the id already exist or not in premium member. If valid then premium charge makes the payment using `payDueAmount()` method and implement discount in the premium member.

- `addUpgradePlan()`: This method allow the regular member to upgrade the plan. Firstly it check the input id is valid or not using `isValid` method. It checks whether the member has met the attendance requirement for the upgrade. If this condition match then this method select new plan. Then it attempts the `upgradePlan` method and display the outcome through the dialog box.
- `addPayDueAmount()`: This method allows a premium member to pay the remaining membership charge. Firstly it check the input id and check is this is valid or not. If valid then it receive the premium charge amount from the input field and process the payment using `payDueAmount` method. If all the process run successfully a confirmation dialog is shown.
- `saveToFile()`: This method take all the information from gym members and store the data in "Members.txt" file. Firstly it write the header with in an formatted columns. And lastly write all the details of premium member and regular member in a well structured row. For regular member id, name, contact, membership start date, plan, price, attendance, loyalty points and active status is saved in the txt file. And for premium member all the regular member details and and additionally payment status, discount amount, paid amount and net amount are included.
- `readFromFile()`: This method reads and display members of gym member stored in the Members.txt file. It ensures that saved member data can be viewed directly from the file in a readable format.

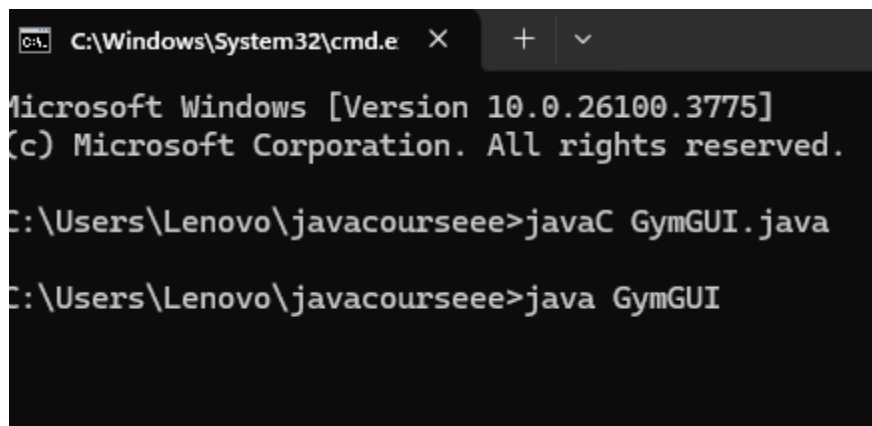
8 Testing

8.1 Test 1

Compile and run the program using command prompt

PROCESS	OUTPUT
Objective	To Compile and Run program using from Cmd.
Action	Opened Command Prompt, Compiled the code using JavaC prompt then the code was executed.
Excepted Output	The GymGUI program should Run.
Actual Output	GymGUI program run successfully.
Result	The test was successful.

Table 6:Test 1



```

C:\Windows\System32\cmd.e  X  +  v

Microsoft Windows [Version 10.0.26100.3775]
(c) Microsoft Corporation. All rights reserved.

C:\Users\Lenovo\javacourseeee>javaC GymGUI.java

C:\Users\Lenovo\javacourseeee>java GymGUI
  
```

Figure 4: Test 1

Gym Management System

GYM MANAGEMENT SYSTEM

Member Information

Id:

Name:

Location:

Phone:

Email:

Gender ☐ Male ☐ Female ☐ ther

DOB:

Membership Details:

Membership Start:

Referral Source

Paid Amount:

Removal Reason:

Trainer Name:

Membership Plan:

Discount Amount:

Regular Plan Price

Premium Plan Price

Add Regular Member

Deactivate Membership

Revert Premium Member

Calculate Discount

Save to File

Add Premium Member

Mark Attendance

Display Members

Upgrade Plan

Read from File

Activate Membership

Revert Regular Member

Clear Fields

Pay Due Amount

Figure 5: Test 1 Output

8.2 Test 2

Add Regular Member and Premium Member

PROCESS	OUTPUT
Objective	Adding Regular member and premium member respectively.
Action	Regular Member was added first and then premium member was added.
Excepted Output	Both type of members should be added successfully.
Actual Output	Both type of members was added successfully.
Result	The Test was Successful.

Table 7:Test 2

The screenshot displays the 'GYM MANAGEMENT SYSTEM' interface. It features two main sections: 'Member Information' and 'Membership Details'. The 'Member Information' section includes fields for Id (67), Name (manoj), Location (dharan), Phone (9804054003), Email (katwalmanoj67@gmail.com), Gender (Male), and DOB (2025). The 'Membership Details' section includes fields for Membership Start (2025), Referral Source (sujan sir), Paid Amount, Removal Reason, Trainer Name, and a dropdown for Plan (basic). A success message dialog box is overlaid on the form, stating 'Successfully added to Regular Member!'. Below the form, there are several buttons for managing members, including 'Add Regular Member', 'Add Premium Member', 'Activate Membership', 'Deactivate Membership', 'Mark Attendance', 'Revert Regular Member', 'Revert Premium Member', 'Display Members', 'Clear Fields', 'Calculate Discount', 'Upgrade Plan', 'Pay Due Amount', 'Save to File', and 'Read from File'.

Figure 6:Test 2 Output

Gym Management System

GYM MANAGEMENT SYSTEM

Member Information
Id:
Name:
Location:
Phone:
Email:
Gender: ☒ Male
DOB:

Membership Details:
Membership Start:
Referral Source:
Paid Amount:
Removal Reason:
Trainer Name:
Plan:
6500
Premium Plan Price **50000**

Success

 Successfully added to Premium Members!

OK

Add Regular Member

Add Premium Member

Activate Membership

Deactivate Membership

Mark Attendance

Revert Regular Member

Revert Premium Member

Display Members

Clear Fields

Calculate Discount

Upgrade Plan

Pay Due Amount

Save to File

Read from File

Figure 7: Test 2 Output

8.3 Test 3

8.3.1 Test case for mark Attendance

PROCESS	OUTPUT
Objective	Test Case for Mark Attendance and upgrade plan
Action	Regular Member was added first then membership was activated and then attendance was marked and then displayed the member details.
Excepted Output	The added member attendance should be marked and should be displayed 1.
Actual Output	The added member attendance was marked and displayed as 1
Result	The Test was Successful.

Table 8: Test 3

Figure 8: Test 3 Output

Member Information

Id: 123

Name: manoj katwal

Location: dharan

Phone: 9804064002

Email: katwalmanoj67@gmail.com

Gender: ☒ Male ☐ Female

DOB: 2023-4-2

Membership Details:

Membership Start: 2024-2-3

Referral Source: rahul

Paid Amount:

Removal Reason:

Trainer Name:

basic

8500

50000

Premium Plan Price

Add Regular Member Add Premium Member Activate Members

Deactivate Membership Mark Attendance Revert Regular Member

Revert Premium Member Display Members Clear Fields

Calculate Discount Upgrade Plan Pay Due Amount

Save to File Read from File

Figure 9:Test 3 activate membership

--- REGULAR MEMBER ---

ID: 123

Name: manoj katwal

Location: dharan

Phone: 9804064002

Email: katwalmanoj67@gmail.com

Gender: male

DOB: 2023-4-2

Start Date: 2024-2-3

Attendance: 1

Loyalty Points: 5.0

Active Status: true

Plan: basic

Price: 6500.0



Figure 10:Test 3 mark Attendance

8.3.2 Test case for Upgrade plan

PROCESS	OUTPUT
Objective	To upgrade the plan of regular member
Action	Fill all the required fields and add regular member and set membership active and the attendance should be more than 30
Excepted Output	The plan may be selected to standard.
Actual Output	The plan selected to standard.
Result	The test was successful.

Table 9: Test 3 upgrade plan

GYM MANAGEMENT SYSTEM

Member Information

Id: 123

Name: manojkatwal

Location: dharan

Phone: 9804064003

Email: katwalmanoj67@gmail.com

Gender: ☒ Male

DOB: 2025 1

Membership Details:

Membership Start: 2025 1 1

Referral Source: rahul

Paid Amount:

Removal Reason:

Trainer Name: diken

Plan: standard

Premium Plan Price: 6500, 50000

Buttons:

- Add Regular Member
- Add Premium Member
- Activate Membership
- Deactivate Membership
- Mark Attendance
- Revert Regular Member
- Revert Premium Member
- Display Members
- Clear Fields
- Calculate Discount
- Upgrade Plan
- Pay Due Amount
- Save to File
- Read from File

Success Dialog: Successfully added to Regular Member! OK

Figure 11: test 3 adding regular member for upgrade plan

GYM MANAGEMENT SYSTEM

Member Information

Id:

Name:

Location:

Phone:

Email:

Gender: ☒ Male ☐ Female

DOB:

Membership Details:

Membership Start:

Referral Source:

Paid Amount:

Removal Reason:

Trainer Name:

Premium Plan Price

Add Regular Member

Add Premium Member

Activate Membership

Deactivate Membership

Mark Attendance

Revert Regular Member

Revert Premium Member

Display Members

Clear Fields

Calculate Discount

Upgrade Plan

Pay Due Amount

Save to File

Read from File

Figure 12: Test 3 active membership for upgrade plan

Member List

REGULAR MEMBER ---

Id: 123

name: manojkatwal

location: dharan

phone: 9804064003

mail: katwalmanoj67@gmail.com

gender: male

DOB: 2025-1-1

last Date: 2025-1-1

Attendance: 38

overall Points: 190.0

drive Status: true

plan: basic

price: 6500.0

Membership Details:

Membership Start:

Referral Source:

Paid Amount:

Removal Reason:

Trainer Name:

Deactivate Membership

Mark Attendance

Revert Regular Member

Revert Premium Member

Display Members

Clear Fields

Calculate Discount

Upgrade Plan

Pay Due Amount

Save to File

Read from File

Figure 13: Test 3 marking attendance 38 for upgrade plan

Member Information

Id:

Name:

Location:

Phone:

Email:

Gender: ☒ Male ☐ Female

DOB:

Membership Details:

Membership Start:

Referral Source:

Paid Amount:

Removal Reason:

Trainer Name:

Plan:

Regular Plan Price:

Premium Plan Price:

Add Regular Member

Deactivate Membership

Revert Premium Member

Calculate Discount

Save to File

Add Premium Member

Mark Attendance

Display Members

Upgrade Plan

Read from File

Activate Membership

Revert Regular Member

Clear Fields

Pay Due Amount

Figure 14:test 3 successfully upgrade the plan

8.4 Test 4

8.4.1 Test case for calculate discount

PROCESS	OUTPUT
Objective	To calculate the discount for premium member.
Action	Fill all the required fill and add to premium member and paid total amount to get discount
Excepted output	May give 10 percent discount
Actual Output	Give 10 percent discount
Result	The test was successful.

GYM MANAGEMENT SYSTEM

Member Information

Id: 1234

Name: manoj

Location: dharan

Phone: 9804064003

Email: katwaimanoj67@gmail.com

Gender: ☒ Male

DOB: 2025

Membership Details:

Membership Start: 2025 1 1

Referral Source: dhanu

Paid Amount: 50000

Removal Reason:

Trainer Name: rahul

Premium Plan Price: basic

Success

Successfully added to Premium Members!

OK

Buttons:

- Add Regular Member
- Add Premium Member
- Activate Membership
- Deactivate Membership
- Mark Attendance
- Revert Regular Member
- Revert Premium Member
- Display Members
- Clear Fields
- Calculate Discount
- Upgrade Plan
- Pay Due Amount
- Save to File
- Read from File

Figure 15: Test 4 adding premium member for discount

GYM MANAGEMENT SYSTEM

Member Information

Id:

Name:

Location:

Phone:

Email:

Gender: ☒ Male ☐ Female

DOB:

Membership Details:

Membership Start:

Referral Source:

Paid Amount:

Removal Reason:

Trainer Name:

Membership Plan:

Discount Amount:

Regular Plan Price:

Premium Plan Price:

Add Regular Member

Deactivate Membership

Revert Premium Member

Calculate Discount

Save to File

Add Premium Member

Mark Attendance

Display Members

Upgrade Plan

Read from File

Activate Membership

Revert Regular Member

Clear Fields

Pay Due Amount

Figure 16: Test 4 output of discount amount

8.4.2 Test case for pay due amount

PROCESS	OUTPUT
Objective	To test the premium member can successfully pay the due premium amount.

Action	Fill all the required fields, registers a premium member and click the pay due amount.
Excepted output	May paid due amount
Actual Output	The paid due amount has been paid
Result	The test was successful.

Table 10: test case for pay due amount

GYM MANAGEMENT SYSTEM

Member Information

Id: 123

Name: manoj

Location: dharan

Phone: 9804064003

Email: katwalmanoj67@gmail.com

Gender: ☒ Male

DOB: 2025

Membership Details:

Membership Start: 2025 1 1

Referral Source: rahul

Paid Amount: 50000

Removal Reason:

Trainer Name: karki

Plan: basic

Premium Plan Price: 50000

Buttons:

- Add Regular Member
- Add Premium Member
- Activate Membership
- Deactivate Membership
- Mark Attendance
- Revert Regular Member
- Revert Premium Member
- Display Members
- Clear Fields
- Calculate Discount
- Upgrade Plan
- Pay Due Amount
- Save to File
- Read from File

Success Message: Successfully added to Premium Members!

Figure 17: test case of adding premium member for pay due amount

gym Management System

GYM MANAGEMENT SYSTEM

Member Information

Id:

Name:

Location:

Phone:

Email:

Gender: ☒ Male ☐ Female

DOB:

Membership Details:

Membership Start:

Referral Source:

Paid Amount:

Removal Reason:

Trainer Name:

Plan:

Premium Plan Price:

Success

Successfully paid the due amount!

OK

Add Regular Member

Add Premium Member

Activate Membership

Deactivate Membership

Mark Attendance

Revert Regular Member

Revert Premium Member

Display Members

Clear Fields

Calculate Discount

Upgrade Plan

Pay Due Amount

Save to File

Read from File

Figure 18: Test case for pay due amount output

8.4.3 Test case for revert regular member

PROCESS	OUTPUT
Objective	To verify to revert regular member
Action	Fill all the required fields, and add a regular member, then enter the revert regular member button.
Excepted Output	The member should be successfully reverted.
Actual Output	The member successfully reverted.
Result	The test was successful.

Table 11: Test case for revert regular member

The screenshot shows the 'GYM MANAGEMENT SYSTEM' interface. The 'Member Information' section contains fields for Id (1234), Name (manoj), Location (Itahari), Phone (9804064003), Email (katwalmanoj67@gmail.com), Gender (Male), and DOB (2025, 1). The 'Membership Details' section includes Membership Start (2025, 1, 1), Referral Source (sujal), Paid Amount, Removal Reason, Trainer Name (rahul), and Plan (basic). A success message dialog box is displayed over the form, stating 'Successfully added to Regular Member!'. Below the form is a grid of buttons for various actions: Add Regular Member, Add Premium Member, Activate Membership, Deactivate Membership, Mark Attendance, Revert Regular Member, Revert Premium Member, Display Members, Clear Fields, Calculate Discount, Upgrade Plan, Pay Due Amount, Save to File, and Read from File.

Figure 19: Adding regular member for regular member test case

Gym Management System

GYM MANAGEMENT SYSTEM

Member Information

Id:

Name:

Location:

Phone:

Email:

Gender: ☒ Male ☐ Female

DOB:

Membership Details:

Membership Start:

Referral Source:

Paid Amount:

Removal Reason:

Trainer Name:

Plan:

Premium Plan Price:

Success

 Regular member reverted successfully!

OK

Add Regular Member

Add Premium Member

Activate Membership

Deactivate Membership

Mark Attendance

Revert Regular Member

Revert Premium Member

Display Members

Clear Fields

Calculate Discount

Upgrade Plan

Pay Due Amount

Save to File

Read from File

Figure 20: Test case for revert regular member output

8.4.4 Test case for Premium Member

PROCESS	OUTPUT
Objective	To verify revert Premium Member
Action	Fill all the required field and add a premium member then enter the revert premium member button.
Excepted Output	The member should be successful reverted.
Actual Output	The member successful reverted.
Result	The test was successful.

Table 12: Test case for premium member

Gym Management System

GYM MANAGEMENT SYSTEM

Member Information

Id: 1234

Name: manoj

Location: dharan

Phone: 9804064003

Email: katwalmanoj67@gmail.com

Gender: ☒ Male

DOB: 2025

Membership Details:

Membership Start: 2025 1 1

Referral Source: rahul

Paid Amount: 50000

Removal Reason: moneyproblem

Trainer Name: katwaldaju

Premium Plan Price: 50000

Success

Successfully added to Premium Members!

OK

Buttons:

- Add Regular Member
- Add Premium Member
- Activate Membership
- Deactivate Membership
- Mark Attendance
- Revert Regular Member
- Revert Premium Member
- Display Members
- Clear Fields
- Calculate Discount
- Upgrade Plan
- Pay Due Amount
- Save to File
- Read from File

Figure 21: Adding premium member for test case for premium member

Gym Management System

GYM MANAGEMENT SYSTEM

Member Information

Id:

Name:

Location:

Phone:

Email:

Gender: ☒ Male ☐ Female

DOB:

Membership Details:

Membership Start:

Referral Source:

Paid Amount:

Removal Reason:

Trainer Name:

Plan:

Premium Plan Price:

Success

 Premium member reverted successfully!

OK

Add Regular Member

Add Premium Member

Activate Membership

Deactivate Membership

Mark Attendance

Revert Regular Member

Revert Premium Member

Display Members

Clear Fields

Calculate Discount

Upgrade Plan

Pay Due Amount

Save to File

Read from File

Figure 22: Test case for revert regular member

8.5 Test case for save to file

PROCESS	OUTPUT
Objective	To verify the system correctly saved the details of member
Action	Add both premium member and regular member and click the save to file button
Excepted Output	A file name should be created and all the data should be stored there.
Actual Output	A file created and stored all the data there
Action	The test was successfull

Table 13: Test case for save to file

The screenshot shows the 'GYM MANAGEMENT SYSTEM' interface. The 'Member Information' section contains fields for Id (123), Name (manoj), Location (dharan), Phone (9804064003), Email (katwalmanoj57@gmail.com), Gender (Male), and DOB (2025, 1). The 'Membership Details' section includes Membership Start (2025, 1, 1), Referral Source (rahu), Paid Amount, Removal Reason, Trainer Name (katuwal daju), and Plan (basic). A success message dialog box is displayed over the form, stating 'Successfully added to Regular Member!'. Below the form are buttons for 'Add Regular Member', 'Add Premium Member', 'Activate Membership', 'Deactivate Membership', 'Mark Attendance', 'Revert Regular Member', 'Revert Premium Member', 'Display Members', 'Clear Fields', 'Calculate Discount', 'Upgrade Plan', 'Pay Due Amount', 'Save to File', and 'Read from File'.

Figure 23: Adding regular member for test case of save to file button

GYM MANAGEMENT SYSTEM

Member Information

Id:

Name:

Location:

Phone:

Email:

Gender: ☒ Male ☐ Female

DOB:

Membership Details:

Membership Start:

Referral Source:

Paid Amount:

Removal Reason:

Trainer Name:

Plan:

Premium Plan Price:

Success

Successfully added to Premium Members!

OK

Add Regular Member

Add Premium Member

Activate Membership

Deactivate Membership

Mark Attendance

Revert Regular Member

Revert Premium Member

Display Members

Clear Fields

Calculate Discount

Upgrade Plan

Pay Due Amount

Save to File

Read from File

Figure 24: Adding premium member for test case for save to file

Gym Management System

GYM MANAGEMENT SYSTEM

Member Information

Id:

Name:

Location:

Phone:

Email:

Gender: ☒ Male ☐ Female

DOB:

Membership Details:

Membership Start:

Referral Source:

Paid Amount:

Removal Reason:

Trainer Name:

Plan:

Premium Plan Price:

Success

 Members saved to file successfully!

OK

Add Regular Member

Add Premium Member

Activate Membership

Deactivate Membership

Mark Attendance

Revert Regular Member

Revert Premium Member

Display Members

Clear Fields

Calculate Discount

Upgrade Plan

Pay Due Amount

Save to File

Read from File

Figure 25:Click save to file button

8.5.1 Read from file

PROCESS	OUTPUT
Objective	To read the file from the text file.
Action	Add both regular member and premium member , click save to file button and lastly click read from file button
Excepted Output	May stored the data can be seen.
Actual Output	Stored data can be seen.
Result	The test was successful.

Table 14:Test case of Read From file

The screenshot shows the 'GYM MANAGEMENT SYSTEM' interface. The 'Member Information' section contains fields for Id (123), Name (manoj), Location (dharan), Phone (9804064003), Email (katwalmanoj67@gmail.com), Gender (Male), and DOB (2025, 1). The 'Membership Details' section contains fields for Membership Start (2025, 1, 1), Referral Source (rahul), Paid Amount, Removal Reason, Trainer Name (katuwal daju), and a dropdown for Plan (basic). A success message dialog box is displayed over the form, stating 'Successfully added to Regular Member!'. Below the form are buttons for 'Add Regular Member', 'Add Premium Member', 'Activate Membership', 'Deactivate Membership', 'Mark Attendance', 'Revert Regular Member', 'Revert Premium Member', 'Display Members', 'Clear Fields', 'Calculate Discount', 'Upgrade Plan', 'Pay Due Amount', 'Save to File', and 'Read from File'.

Figure 26:Adding regular member for read from file test

GYM MANAGEMENT SYSTEM			
Member Information		Membership Details:	
Id:	1234	Membership Start:	2025 1 1
Name:	manoj	Referral Source	rahul
Location:	dharan	Paid Amount:	
Phone:	9804064003	Removal Reason:	
Email:	katwalmanoj67@gmail.com	Trainer Name:	katuwaldaju
Gender	☒ Male ☐ Female		basic
DOB:	2025 1 1		6500 50000
		Premium Plan Price	

Add Regular Member	Add Premium Member	Activate Membership
Deactivate Membership	Mark Attendance	Revert Regular Member
Revert Premium Member	Display Members	Clear Fields
Calculate Discount	Upgrade Plan	Pay Due Amount
Save to File	Read from File	

Figure 27: Adding premium member for test case for read from file

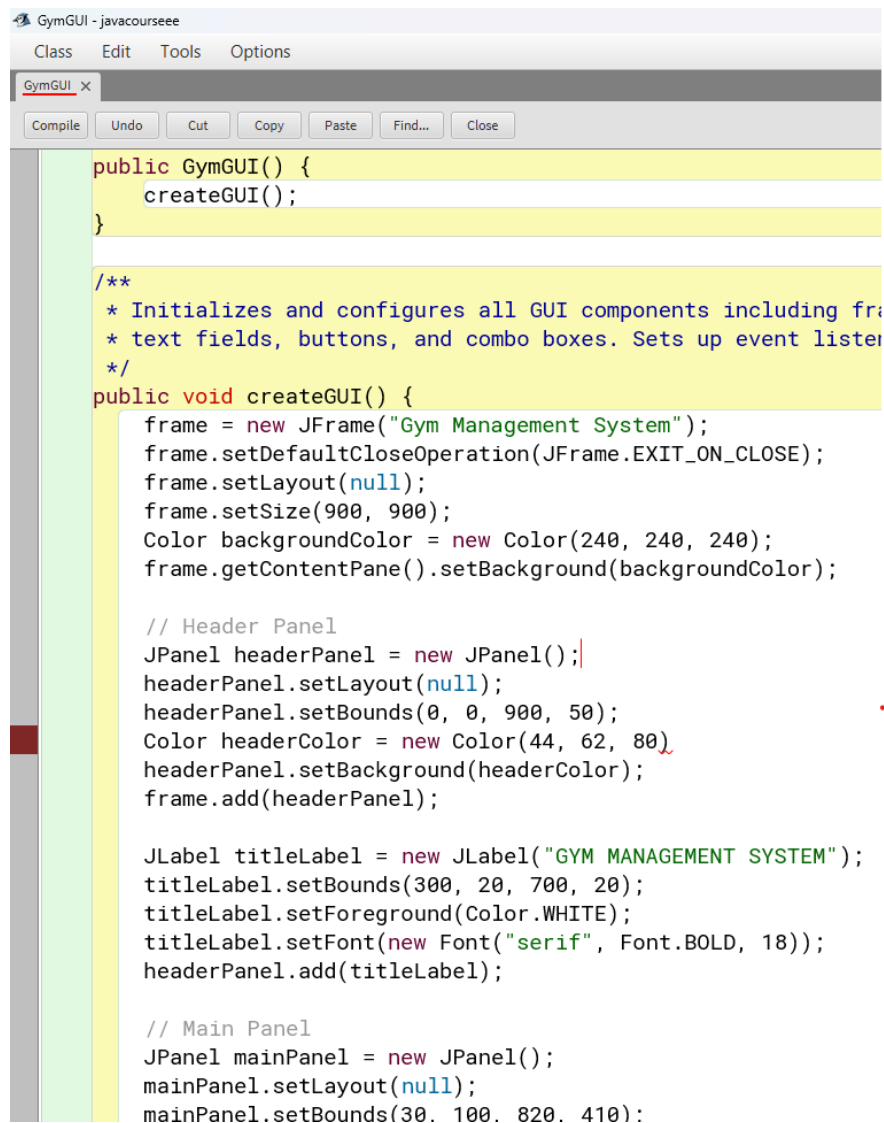
Gym Member Details													
ID	Name	Location	Phone	Email	Membership Start Date	Plan	Price	Attendance	Loyalty Points	Active	Status	Full Payment	Discount
123	manoj	dharan	9804064003	katwalmanoj67@gmail.com	2025-1-1	basic	6500.00	0	0.00	false	N/A	N/A	N/A
1234	manoj	dharan	9804064003	katwalmanoj67@gmail.com	2025-1-1	Premium	N/A	0	0.00	false	false	0.00	0.0

Figure 28: Output of test case of read from file

9 Error detection and correction

9.1 Syntax error

Syntax error in java means mistake of programmer while writing the code occurs when compiler cannot understand the source code



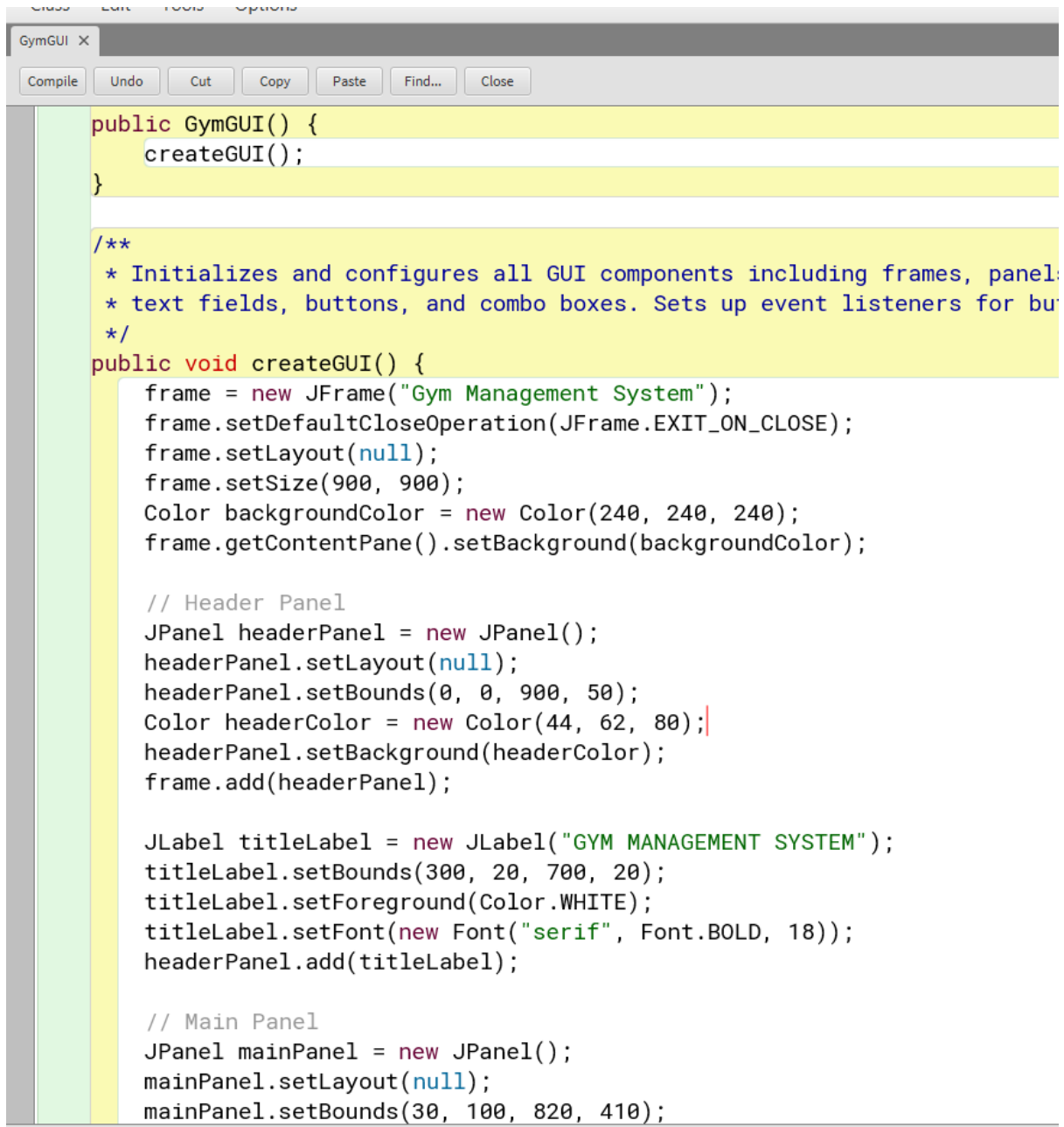
The screenshot shows a Java IDE window titled "GymGUI - javacourseee". The menu bar includes "Class", "Edit", "Tools", and "Options". The toolbar contains buttons for "Compile", "Undo", "Cut", "Copy", "Paste", "Find...", and "Close". The code editor displays the following Java code:

```
public GymGUI() {  
    createGUI();  
}  
  
/**  
 * Initializes and configures all GUI components including fra  
 * text fields, buttons, and combo boxes. Sets up event listen  
 */  
public void createGUI() {  
    frame = new JFrame("Gym Management System");  
    frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);  
    frame.setLayout(null);  
    frame.setSize(900, 900);  
    Color backgroundColor = new Color(240, 240, 240);  
    frame.getContentPane().setBackground(backgroundColor);  
  
    // Header Panel  
    JPanel headerPanel = new JPanel();  
    headerPanel.setLayout(null);  
    headerPanel.setBounds(0, 0, 900, 50);  
    Color headerColor = new Color(44, 62, 80);  
    headerPanel.setBackground(headerColor);  
    frame.add(headerPanel);  
  
    JLabel titleLabel = new JLabel("GYM MANAGEMENT SYSTEM");  
    titleLabel.setBounds(300, 20, 700, 20);  
    titleLabel.setForeground(Color.WHITE);  
    titleLabel.setFont(new Font("serif", Font.BOLD, 18));  
    headerPanel.add(titleLabel);  
  
    // Main Panel  
    JPanel mainPanel = new JPanel();  
    mainPanel.setLayout(null);  
    mainPanel.setBounds(30, 100, 820, 410);
```

A red squiggly line under the closing curly brace of the `createGUI()` method indicates a syntax error. The IDE's background has a light green vertical bar on the left side.

Figure 29: Detecting syntax error

(Burnham, 2025)



```

GymGUI X
Compile Undo Cut Copy Paste Find... Close

public GymGUI() {
    createGUI();
}

/**
 * Initializes and configures all GUI components including frames, panels,
 * text fields, buttons, and combo boxes. Sets up event listeners for buttons.
 */
public void createGUI() {
    JFrame frame = new JFrame("Gym Management System");
    frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    frame.setLayout(null);
    frame.setSize(900, 900);
    Color backgroundColor = new Color(240, 240, 240);
    frame.getContentPane().setBackground(backgroundColor);

    // Header Panel
    JPanel headerPanel = new JPanel();
    headerPanel.setLayout(null);
    headerPanel.setBounds(0, 0, 900, 50);
    Color headerColor = new Color(44, 62, 80);
    headerPanel.setBackground(headerColor);
    frame.add(headerPanel);

    JLabel titleLabel = new JLabel("GYM MANAGEMENT SYSTEM");
    titleLabel.setBounds(300, 20, 700, 20);
    titleLabel.setForeground(Color.WHITE);
    titleLabel.setFont(new Font("serif", Font.BOLD, 18));
    headerPanel.add(titleLabel);

    // Main Panel
    JPanel mainPanel = new JPanel();
    mainPanel.setLayout(null);
    mainPanel.setBounds(30, 100, 820, 410);

```

Figure 30: Solving syntax error

9.2 Run time error

Run time error means that error when a program doesn't contain any syntax error but it occurs during the execution of the program.

```
* Displays error messages for invalid inputs or duplicate IDs.
*/
public void addRegularMember() {

    int id = Integer.parseInt(idField.getText());
    if (isValidId(id) != null) {
        JOptionPane.showMessageDialog(frame, "The id already exists, please enter another id.", "Warning", JOptionPane.WARNING_MESSAGE);
        return;
    }

    String name = nameField.getText();
    String location = locationField.getText();
    String phone = phoneField.getText();
    String email = emailField.getText();
    String referral = referralField.getText();

    String gender = "";
    if (maleButton.isSelected()) {
        gender = "male";
    } else if (femaleButton.isSelected()) {
        gender = "female";
    } else if (otherButton.isSelected()) {
        gender = "other";
    } else {
        JOptionPane.showMessageDialog(frame, "Please select a gender!", "Error", JOptionPane.ERROR_MESSAGE);
        return;
    }

    if (dobYearBox.getSelectedIndex() == -1 || dobMonthBox.getSelectedIndex() == -1 || dobDayBox.getSelectedIndex() == -1 ||
        msYearBox.getSelectedIndex() == -1 || msMonthBox.getSelectedIndex() == -1 || msDayBox.getSelectedIndex() == -1) {
        JOptionPane.showMessageDialog(frame, "Please select valid dates for DOB and Membership Start!", "Error", JOptionPane.ERROR_MESSAGE);
        return;
    }
}
```

Figure 31: Detection of run time error in code

The screenshot shows a Java Swing application titled "GYM MANAGEMENT SYSTEM". It features two main panels: "Member Information" and "Membership Details". The "Member Information" panel contains text fields for Id, Name, Location, Phone, and Email, radio buttons for Gender (Male, Female, Other), and dropdown menus for DOB (Year, Month, Day). The "Membership Details" panel contains dropdown menus for Membership Start (Year, Month, Day), a text field for Referral Source, text fields for Paid Amount and Removal Reason, a text field for Trainer Name, a dropdown for Membership Plan, and text fields for Discount Amount, Regular Plan Price, and Premium Plan Price. Below these panels are buttons for "Add Regular Member", "Add Premium Member", "Activate Membership", "Deactivate Membership", "Mark Attendance", "Revert Regular Member", "Revert Premium Member", "Display Members", "Clear Fields", "Calculate Discount", "Upgrade Plan", "Pay Due Amount", "Save to File", and "Read from File". On the right side of the GUI, a Java error stack trace is visible, indicating a run-time error.

Figure 32: Detection of run time error in GUI

```

*/
public void addRegularMember() {
    try {
        int id = Integer.parseInt(idField.getText());
        if (isValidId(id) != null) {
            JOptionPane.showMessageDialog(frame, "The id already exists, please enter another id.", "Warning", JOptionPane.WARNING_MESSAGE);
            return;
        }

        String name = nameField.getText();
        String location = locationField.getText();
        String phone = phoneField.getText();
        String email = emailField.getText();
        String referral = referralField.getText();

        String gender = "";
        if (maleButton.isSelected()) {
            gender = "male";
        } else if (femaleButton.isSelected()) {
            gender = "female";
        } else if (otherButton.isSelected()) {
            gender = "other";
        } else {
            JOptionPane.showMessageDialog(frame, "Please select a gender!", "Error", JOptionPane.ERROR_MESSAGE);
            return;
        }

        if (dobYearBox.getSelectedIndex() == -1 || dobMonthBox.getSelectedIndex() == -1 || dobDayBox.getSelectedIndex() == -1 ||
            msYearBox.getSelectedIndex() == -1 || msMonthBox.getSelectedIndex() == -1 || msDayBox.getSelectedIndex() == -1) {
            JOptionPane.showMessageDialog(frame, "Please select valid dates for DOB and Membership Start!", "Error", JOptionPane.ERROR_MESSAGE);
            return;
        }

        String DOB = dobYearBox.getSelectedItem() + "-" + dobMonthBox.getSelectedItem() + "-" + dobDayBox.getSelectedItem();
    }
}

```

Figure 33:Correction of run time error in code

Figure 34:Correction of run time error in GUI

(Burnham, 2025)

9.3 Logical error

Logical error is a mistake in a source code that result unexpected behavior. It is a type of run-time error it means it occurs during the execution of the program.

```
try {
    int id = Integer.parseInt(idField.getText());
    if (isValidId(id) != null) {
        JOptionPane.showMessageDialog(frame, "The id already exists, please enter another id.", "Warning", JOptionPane.WARNING_MESSAGE);
        return;
    }

    String name = nameField.getText();
    String location = locationField.getText();
    String phone = phoneField.getText();
    String email = emailField.getText();
    String referral = referralField.getText();

    String gender = "";
    if (maleButton.isSelected()) {
        gender = "male";
    } else if (femaleButton.isSelected()) {
        gender = "female";
    } else if (otherButton.isSelected()) {
        gender = "other";
    } else {
        JOptionPane.showMessageDialog(frame, "Please select a gender!", "Error", JOptionPane.ERROR_MESSAGE);
        return;
    }

    if (dobYearBox.getSelectedIndex() == -1 || dobMonthBox.getSelectedIndex() == -1 || dobDayBox.getSelectedIndex() == -1 ||
        msYearBox.getSelectedIndex() == -1 || msMonthBox.getSelectedIndex() == -1 || msDayBox.getSelectedIndex() == -1) {
        JOptionPane.showMessageDialog(frame, "Please select valid dates for DOB and Membership Start!", "Error", JOptionPane.ERROR_MESSAGE);
        return;
    }

    String DOB = dobYearBox.getSelectedItem() + "-" + dobMonthBox.getSelectedItem() + "-" + dobDayBox.getSelectedItem();
    String membershipStartDate = msYearBox.getSelectedItem() + "-" + msMonthBox.getSelectedItem() + "-" + msDayBox.getSelectedItem();

    if (name.isEmpty() || location.isEmpty() || phone.isEmpty() || email.isEmpty() || referral.isEmpty()) {
        JOptionPane.showMessageDialog(frame, "Please fill all the required fields!", "Error", JOptionPane.ERROR_MESSAGE);
        return;
    }

    if (!phone.matches("\\d{10}")) {
        JOptionPane.showMessageDialog(frame, "Phone number must be exactly 10 digits (e.g., 1234567890)!", "Error", JOptionPane.ERROR_MESSAGE);
        return;
    }

    if (!email.matches("^\\w._%+-]+@[\\w.-]+\\.([a-zA-Z]{2,})$")) {
        JOptionPane.showMessageDialog(frame, "Please enter a valid email address (e.g., example@domain.com)!", "Error", JOptionPane.ERROR_MESSAGE);
        return;
    }

    RegularMember member = new RegularMember(id, name, location, phone, email, gender, DOB, membershipStartDate, referral);
    members.add(member);
    JOptionPane.showMessageDialog(frame, "Successfully added to Regular Member!", "Success", JOptionPane.INFORMATION_MESSAGE);
} catch (NumberFormatException nfe) {
    JOptionPane.showMessageDialog(frame, "Please enter a valid ID number!", "Error", JOptionPane.ERROR_MESSAGE);
}
```

Figure 35: Detection of logical error code

```

try {
    int id = Integer.parseInt(idField.getText());
    if (isValidId(id) != null) {
        JOptionPane.showMessageDialog(frame, "The id already exists, please enter another id.", "Warning", JOptionPane.WARNING_MESSAGE);
        return;
    }

    String name = nameField.getText();
    String location = locationField.getText();
    String phone = phoneField.getText();
    String email = emailField.getText();
    String referral = referralField.getText();

    String gender = "";
    if (maleButton.isSelected()) {
        gender = "male";
    } else if (femaleButton.isSelected()) {
        gender = "female";
    } else if (otherButton.isSelected()) {
        gender = "other";
    } else {
        JOptionPane.showMessageDialog(frame, "Please select a gender!", "Error", JOptionPane.ERROR_MESSAGE);
        return;
    }

    if (dobYearBox.getSelectedIndex() == -1 || dobMonthBox.getSelectedIndex() == -1 || dobDayBox.getSelectedIndex() == -1 ||
        msYearBox.getSelectedIndex() == -1 || msMonthBox.getSelectedIndex() == -1 || msDayBox.getSelectedIndex() == -1) {
        JOptionPane.showMessageDialog(frame, "Please select valid dates for DOB and Membership Start!", "Error", JOptionPane.ERROR_MESSAGE);
        return;
    }

    String DOB = dobYearBox.getSelectedItem() + "-" + dobMonthBox.getSelectedItem() + "-" + dobDayBox.getSelectedItem();
    String membershipStartDate = msYearBox.getSelectedItem() + "-" + msMonthBox.getSelectedItem() + "-" + msDayBox.getSelectedItem();

    if (name.isEmpty() || location.isEmpty() || phone.isEmpty() || email.isEmpty() || referral.isEmpty()) {
        JOptionPane.showMessageDialog(frame, "Please fill all the required fields!", "Error", JOptionPane.ERROR_MESSAGE);
        return;
    }

    if (!phone.matches("\\d{10}")) {
        JOptionPane.showMessageDialog(frame, "Phone number must be exactly 10 digits (e.g., 1234567890)!", "Error", JOptionPane.ERROR_MESSAGE);
        return;
    }

    if (!email.matches("^\\w._%+-]+@[\\w.-]+\\.([a-zA-Z]{2,})$")) {
        JOptionPane.showMessageDialog(frame, "Please enter a valid email address (e.g., example@domain.com)!", "Error", JOptionPane.ERROR_MESSAGE);
        return;
    }

    RegularMember member = new RegularMember(id, name, location, phone, email, gender, DOB, membershipStartDate, referral);
    members.add(member);
    JOptionPane.showMessageDialog(frame, "Successfully added to Regular Member!", "Success", JOptionPane.INFORMATION_MESSAGE);
} catch (NumberFormatException nfe) {
    JOptionPane.showMessageDialog(frame, "Please enter a valid ID number!", "Error", JOptionPane.ERROR_MESSAGE);
}
}

```

Figure 36:Correcting the logical error

10 Conclusion

In this project, I successfully developed a Gym Management System using Java and using object oriented principal concept. The system includes one parent class named Gym Members and sub class premium member and regular member. I have also created a GUI using java swing.

While working on this assignment I have learned how to use Java concept like abstraction, inheritance, polymorphism. I have also work with Array List and also create a user friendly GUI using Java Swing and also implement all the exception in the program.

During the development of the system I have faced several issues like handling user input fields, handling errors, and mainly I have struggled to organize the components using layout manager to null.

To solve the problem I start debugging the program, used try-catch block for error handling and for organize components I took a notes copy and first run dry program in my notes copy which helps me to organize component easily.

Overall the project was a great learning experience that build by Java Learning experience that build my java knowledge in a real word scenario.

11 References

Burnham, D., 2025. */tutorial/david/errors-in-java.* [Online]
Available at: <https://codehs.com/tutorial/david/errors-in-java>
[Accessed 15 05 2025].

Burnham, D., 2025. *tutorial/david/errors-in-java.* [Online]
Available at: <https://codehs.com/tutorial/david/errors-in-java>
[Accessed 15 05 2025].

Sovereign Corporate Tower, 2025. *how-to-write-a-pseudo-code/.* [Online]
Available at: <https://www.geeksforgeeks.org/how-to-write-a-pseudo-code/>
[Accessed 15 05 2025].

Sovereign Corporate Tower, 2025. *unified-modeling-language-uml-class-diagrams/.*
[Online]
Available at: <https://www.geeksforgeeks.org/unified-modeling-language-uml-class-diagrams/>
[Accessed 15 05 2025].

12 Appendix

1. Gym Member

/**

* Abstract class representing a generic gym member.

* It holds the basic details and functionalities shared by all types of gym members.

*

* @author Manoj Katwal

*/

```
public abstract class GymMember {  
    protected int id;  
  
    protected String name;  
  
    protected String location;  
  
    protected String phone;  
  
    protected String email;  
  
    protected String gender;  
  
    protected String DOB;  
  
    protected String membershipStartDate;  
  
    protected int attendance;  
  
    protected double loyaltyPoints;  
  
    protected boolean activeStatus;  
  
    /**
```

* Constructs a GymMember with the provided personal and membership information.

*

* @param id Unique member ID

* @param name Member's full name

* @param location Member's location or address

* @param phone Member's phone number

* @param email Member's email address

* @param gender Member's gender

* @param DOB Member's date of birth

* @param membershipStartDate Date when the membership started

*/

```
public GymMember(int id, String name, String location, String phone, String email,
```

```
    String gender, String DOB, String membershipStartDate) {
```

```
    this.id = id;
```

```
    this.name = name;
```

```
    this.location = location;
```

```
    this.phone = phone;
```

```
    this.email = email;
```

```
    this.gender = gender;
```

```
    this.DOB = DOB;
```

```
    this.membershipStartDate = membershipStartDate;
```

```
    this.attendance = 0;
```

```
    this.loyaltyPoints = 0;

    this.activeStatus = false;
}
```

```
// Getter methods for each attribute
```

```
public int getId() { return id; }

public String getName() { return name; }

public String getLocation() { return location; }

public String getPhone() { return phone; }

public String getEmail() { return email; }

public String getGender() { return gender; }

public String getDOB() { return DOB; }

public String getMembershipStartDate() { return membershipStartDate; }

public int getAttendance() { return attendance; }

public double getLoyaltyPoints() { return loyaltyPoints; }

public boolean getActiveStatus() { return activeStatus; }
```

```
/**
```

```
 * Abstract method to be implemented by subclasses to define how attendance is
marked.
```

```
*/
```

```
public abstract void markAttendance();
```

```
/**  
 * Activates the member's gym membership.  
 */  
public void activateMembership() {  
    this.activeStatus = true;  
}  
  
/**  
 * Deactivates the member's gym membership if currently active.  
 */  
public void deactivateMembership() {  
    if (this.activeStatus) {  
        this.activeStatus = false;  
    }  
}  
  
/**  
 * Resets the member's status, attendance, and loyalty points.  
 */  
public void resetMember() {  
    this.activeStatus = false;  
    this.attendance = 0;  
    this.loyaltyPoints = 0;
```

```
}

/**
 * Displays all the information of the gym member.
 */
public void display() {
    System.out.println("Member ID: " + id);
    System.out.println("Name: " + name);
    System.out.println("Location: " + location);
    System.out.println("Phone: " + phone);
    System.out.println("Email: " + email);
    System.out.println("Gender: " + gender);
    System.out.println("DOB: " + DOB);
    System.out.println("Membership Start Date: " + membershipStartDate);
    System.out.println("Attendance: " + attendance);
    System.out.println("Loyalty Points: " + loyaltyPoints);
    System.out.println("Active Status: " + activeStatus);
}
}
```

2. Regular Member

```

/**
 * Class representing a regular gym member.
 * A regular member has access to a plan with a fixed price and may upgrade
 after reaching a certain attendance threshold.
 * This class extends the abstract class GymMember.
 *
 * @author Manoj Katwal
 */
public class RegularMember extends GymMember {
    private final int attendanceLimit = 30;
    private boolean isEligibleForUpgrade;
    private String removalReason;
    private String referralSource;
    private String plan;
    private double price;

    /**
     * Constructs a RegularMember with the provided personal and referral
 information.
     *
     * @param id Unique member ID
     * @param name Member's full name
     * @param location Member's location
     * @param phone Member's phone number
     * @param email Member's email address
     * @param gender Member's gender
     * @param DOB Member's date of birth
     * @param membershipStartDate Date when the membership started
     * @param referralSource Source through which the member was referred
     */
    public RegularMember(int id, String name, String location, String phone, String
 email,
                        String gender, String DOB, String membershipStartDate, String
 referralSource) {
        super(id, name, location, phone, email, gender, DOB,
 membershipStartDate);
        this.isEligibleForUpgrade = false;
        this.removalReason = "";
        this.referralSource = referralSource;
        this.plan = "basic";
        this.price = 6500;
    }
}

```

```
// Getter methods
public int getAttendanceLimit() { return attendanceLimit; }
public boolean getIsEligibleForUpgrade() { return isEligibleForUpgrade; }
public String getRemovalReason() { return removalReason; }
public String getReferralSource() { return referralSource; }
public String getPlan() { return plan; }
public double getPrice() { return price; }

/**
 * Increases the attendance and loyalty points by 1 and 5 respectively.
 * Marks the member as eligible for upgrade if attendance reaches the limit.
 */
@Override
public void markAttendance() {
    this.attendance++;
    this.loyaltyPoints += 5;
    if (this.attendance >= attendanceLimit) {
        this.isEligibleForUpgrade = true;
    }
}

/**
 * Returns the price of the specified membership plan.
 *
 * @param plan The membership plan name (basic, standard, deluxe)
 * @return Price of the plan or -1 if the plan is invalid
 */
public double getPlanPrice(String plan) {
    switch(plan.toLowerCase()) {
        case "basic": return 6500;
        case "standard": return 12500;
        case "deluxe": return 18500;
        default: return -1;
    }
}

/**
 * Attempts to upgrade the member's current plan.
 *
 * @param newPlan The new plan to upgrade to
 * @return A message indicating the result of the upgrade attempt
 */
```



```
public String upgradePlan(String newPlan) {
    if (!isEligibleForUpgrade) {
        return "Not eligible for upgrade. Need more attendance.";
    }

    if (newPlan.toLowerCase().equals(this.plan)) {
        return "Already subscribed to this plan.";
    }

    double newPrice = getPlanPrice(newPlan);
    if (newPrice == -1) {
        return "Invalid plan selected.";
    }

    this.plan = newPlan.toLowerCase();
    this.price = newPrice;
    return "Plan upgraded successfully to " + newPlan;
}

/**
 * Reverts the member to the default basic plan and resets their status.
 *
 * @param removalReason The reason for reverting the membership
 */
public void revertRegularMember(String removalReason) {
    super.resetMember();
    this.isEligibleForUpgrade = false;
    this.plan = "basic";
    this.price = 6500;
    this.removalReason = removalReason;
}

/**
 * Displays the member's personal, membership, and plan details.
 */
@Override
public void display() {
    super.display();
    System.out.println("Plan: " + plan);
    System.out.println("Price: " + price);
    if (!removalReason.isEmpty()) {
        System.out.println("Removal Reason: " + removalReason);
    }
}
```

```
    }  
  }  
3. Premium Member  
  
/**  
 * Class representing a premium gym member.  
 * Premium members pay a fixed charge and may receive a discount for full  
payment.  
 * This class extends the abstract class GymMember.  
 *  
 * @author Manoj Katwal  
 */  
  
public class PremiumMember extends GymMember {  
    private final double premiumCharge = 50000;  
    private String personalTrainer;  
    private boolean isFullPayment;  
    private double paidAmount;  
    private double discountAmount;  
  
    /**  
     * Constructs a PremiumMember with the provided details and assigned personal  
trainer.  
     *  
     * @param id Unique member ID  
     * @param name Member's full name  
     * @param location Member's address or location  
     * @param phone Member's phone number  
     * @param email Member's email address  
     * @param gender Member's gender  
     * @param DOB Member's date of birth  
     * @param membershipStartDate Date when the membership began
```

```
* @param personalTrainer Name of the personal trainer assigned
*/

public PremiumMember(int id, String name, String location, String phone, String
email,
    String gender, String DOB, String membershipStartDate, String
personalTrainer) {
    super(id, name, location, phone, email, gender, DOB, membershipStartDate);
    this.personalTrainer = personalTrainer;
    this.isFullPayment = false;
    this.paidAmount = 0;
    this.discountAmount = 0;
}

// Getter methods
public double getPremiumCharge() { return premiumCharge; }
public String getPersonalTrainer() { return personalTrainer; }
public boolean getIsFullPayment() { return isFullPayment; }
public double getPaidAmount() { return paidAmount; }
public double getDiscountAmount() { return discountAmount; }

/**
 * Marks attendance for the premium member and adds loyalty points.
 */
@Override
public void markAttendance() {
    this.attendance++;
    this.loyaltyPoints += 5;
}
```

```
/**
 * Processes a payment towards the premium charge.
 *
 * @param amount The amount being paid
 * @return A message indicating success or error of the payment process
 */
public String payDueAmount(double amount) {
    if (this.isFullPayment) {
        return "Payment is already complete.";
    }

    if (this.paidAmount + amount > premiumCharge) {
        return "Payment amount exceeds the premium charge.";
    }

    this.paidAmount += amount;
    if (this.paidAmount == premiumCharge) {
        this.isFullPayment = true;
    }

    double remainingAmount = premiumCharge - this.paidAmount;
    return "Payment successful. Remaining amount: " + remainingAmount;
}

/**
 * Calculates a 10% discount on the premium charge if the full payment is
 * completed.
 */
public void calculateDiscount() {
```

```
        if (isFullPayment) {
            this.discountAmount = premiumCharge * 0.10;
            System.out.println("Discount calculated: " + discountAmount);
        } else {
            System.out.println("No discount available. Complete payment first.");
        }
    }

    /**
     * Resets the member to default values for re-registration or downgrade purposes.
     */
    public void revertPremiumMember() {
        super.resetMember();
        this.personalTrainer = "";
        this.isFullPayment = false;
        this.paidAmount = 0;
        this.discountAmount = 0;
    }

    /**
     * Displays all member details including premium payment status and discount.
     */
    @Override
    public void display() {
        super.display();
        System.out.println("Personal Trainer: " + personalTrainer);
        System.out.println("Paid Amount: " + paidAmount);
        System.out.println("Payment Status: " + (isFullPayment ? "Complete" :
        "Incomplete"));
    }
```

```
        double remainingAmount = premiumCharge - paidAmount;
        System.out.println("Remaining Amount: " + remainingAmount);
        if (isFullPayment) {
            System.out.println("Discount Amount: " + discountAmount);
        }
    }
}
```

4. Gym GUI

```
import javax.swing.BorderFactory;
import javax.swing.ButtonGroup;
import javax.swing.JComboBox;
import javax.swing.JFrame;
import javax.swing.JLabel;
import javax.swing.JOptionPane;
import javax.swing.JPanel;
import javax.swing.JRadioButton;
import javax.swing.JScrollPane;
import javax.swing.JTextArea;
import javax.swing.JTextField;
import javax.swing.JButton;

import java.awt.Color;
import java.awt.Font;
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;
import java.util.ArrayList;
import java.io.File;
import java.io.FileWriter;
```

```
import java.io.IOException;
import java.util.Scanner;

/**
 * The GymGUI class represents the main graphical user interface for the Gym
 * Management System.
 * It allows users to manage both regular and premium gym members, handle
 * memberships, track attendance,
 * apply discounts, and view member details through a Swing-based interface.
 * @author Manoj Katwal
 */
public class GymGUI {

    JFrame frame;

    JTextField idField, nameField, locationField, phoneField, emailField, referralField;
    JTextField paidField, removalField, trainerField, memberField, reguPriceField,
    preCharField, disAmouField;

    JComboBox<String> dobYearBox, dobMonthBox, dobDayBox;
    JComboBox<String> msYearBox, msMonthBox, msDayBox;
    JComboBox<String> planBox;

    JRadioButton maleButton, femaleButton, otherButton;

    ButtonGroup genderGroup;

    private ArrayList<GymMember> members = new ArrayList<>();
```

```
        JButton regularmembtn, premiumbtn, activateMembershipbtn,  
        deactivateMembershipButtton, markAttendancebtn,  
        revertRegularMemberbtn, revertPremiumbtn, upgradeButton,  
        discountButton;
```

```
/**
```

```
 * Constructs the GymGUI and initializes the graphical interface.
```

```
*/
```

```
public GymGUI() {  
    createGUI();  
}
```

```
/**
```

```
 * Initializes and configures all GUI components including frames, panels, labels,
```

```
 * text fields, buttons, and combo boxes. Sets up event listeners for button actions.
```

```
*/
```

```
public void createGUI() {  
    frame = new JFrame("Gym Management System");  
    frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);  
    frame.setLayout(null);  
    frame.setSize(900, 900);  
    Color backgroundColor = new Color(240, 240, 240);  
    frame.getContentPane().setBackground(backgroundColor);  
  
    // Header Panel  
    JPanel headerPanel = new JPanel();  
    headerPanel.setLayout(null);  
    headerPanel.setBounds(0, 0, 900, 50);  
    Color headerColor = new Color(44, 62, 80);
```



```
headerPanel.setBackground(headerColor);  
frame.add(headerPanel);
```

```
JLabel titleLabel = new JLabel("GYM MANAGEMENT SYSTEM");  
titleLabel.setBounds(300, 20, 700, 20);  
titleLabel.setForeground(Color.WHITE);  
titleLabel.setFont(new Font("serif", Font.BOLD, 18));  
headerPanel.add(titleLabel);
```

```
// Main Panel
```

```
JPanel mainPanel = new JPanel();  
mainPanel.setLayout(null);  
mainPanel.setBounds(30, 100, 820, 410);  
mainPanel.setBackground(Color.WHITE);  
mainPanel.setBorder(BorderFactory.createLineBorder(new Color(200, 200,  
200)));  
frame.add(mainPanel);
```

```
JLabel membInfo = new JLabel("Member Information");  
membInfo.setBounds(10, 10, 380, 25);  
membInfo.setFont(new Font("Segoe UI", Font.BOLD, 16));  
mainPanel.add(membInfo);
```

```
JLabel idLabel = new JLabel("Id:");  
idLabel.setBounds(20, 50, 150, 25);  
mainPanel.add(idLabel);
```

```
idField = new JTextField();  
idField.setBounds(180, 50, 200, 30);
```

```
mainPanel.add(idField);
```

```
JLabel nameLabel = new JLabel("Name:");  
nameLabel.setBounds(20, 90, 150, 30);  
mainPanel.add(nameLabel);
```

```
nameField = new JTextField();  
nameField.setBounds(180, 90, 200, 30);  
mainPanel.add(nameField);
```

```
JLabel locationLabel = new JLabel("Location:");  
locationLabel.setBounds(20, 130, 150, 30);  
mainPanel.add(locationLabel);
```

```
locationField = new JTextField();  
locationField.setBounds(180, 130, 200, 30);  
mainPanel.add(locationField);
```

```
JLabel phoneLabel = new JLabel("Phone:");  
phoneLabel.setBounds(20, 170, 150, 30);  
mainPanel.add(phoneLabel);
```

```
phoneField = new JTextField();  
phoneField.setBounds(180, 170, 200, 30);  
mainPanel.add(phoneField);
```

```
JLabel emailLabel = new JLabel("Email:");  
emailLabel.setBounds(20, 210, 150, 30);
```

```
mainPanel.add(emailLabel);
```

```
emailField = new JTextField();  
emailField.setBounds(180, 210, 200, 30);  
mainPanel.add(emailField);
```

```
JLabel genderLabel = new JLabel("Gender");  
genderLabel.setBounds(20, 250, 150, 30);  
mainPanel.add(genderLabel);
```

```
maleButton = new JRadioButton("Male");  
maleButton.setBounds(180, 250, 60, 30);  
maleButton.setBackground(Color.WHITE);  
mainPanel.add(maleButton);
```

```
femaleButton = new JRadioButton("Female");  
femaleButton.setBounds(250, 250, 100, 30);  
femaleButton.setBackground(Color.WHITE);  
mainPanel.add(femaleButton);
```

```
otherButton = new JRadioButton("Other");  
otherButton.setBounds(320, 250, 100, 30);  
otherButton.setBackground(Color.WHITE);  
mainPanel.add(otherButton);
```

```
genderGroup = new ButtonGroup();  
genderGroup.add(maleButton);  
genderGroup.add(femaleButton);
```

```
genderGroup.add(otherButton);
```

```
JLabel dobLabel = new JLabel("DOB:");  
dobLabel.setBounds(20, 290, 150, 30);  
mainPanel.add(dobLabel);
```

```
dobYearBox = new JComboBox<>(getYears());  
dobYearBox.setBounds(180, 285, 60, 25);  
mainPanel.add(dobYearBox);
```

```
dobMonthBox = new JComboBox<>(getMonths());  
dobMonthBox.setBounds(250, 285, 60, 25);  
mainPanel.add(dobMonthBox);
```

```
dobDayBox = new JComboBox<>(getDays());  
dobDayBox.setBounds(315, 285, 60, 25);  
mainPanel.add(dobDayBox);
```

```
JLabel membershipLabel = new JLabel("Membership Details:");  
membershipLabel.setBounds(440, 10, 380, 25);  
membershipLabel.setFont(new Font("Segoe UI", Font.BOLD, 16));  
mainPanel.add(membershipLabel);
```

```
JLabel membershipstart = new JLabel("Membership Start:");  
membershipstart.setBounds(440, 50, 150, 30);  
mainPanel.add(membershipstart);
```

```
msYearBox = new JComboBox<>(getYears());
```

```
msYearBox.setBounds(600, 50, 60, 25);  
mainPanel.add(msYearBox);
```

```
msMonthBox = new JComboBox<>(getMonths());  
msMonthBox.setBounds(670, 50, 60, 25);  
mainPanel.add(msMonthBox);
```

```
msDayBox = new JComboBox<>(getDays());  
msDayBox.setBounds(750, 50, 60, 25);  
mainPanel.add(msDayBox);
```

```
JLabel referralLabel = new JLabel("Referral Source");  
referralLabel.setBounds(440, 100, 150, 30);  
mainPanel.add(referralLabel);
```

```
referralField = new JTextField();  
referralField.setBounds(600, 100, 200, 30);  
mainPanel.add(referralField);
```

```
JLabel paidLabel = new JLabel("Paid Amount:");  
paidLabel.setBounds(440, 140, 150, 30);  
mainPanel.add(paidLabel);
```

```
paidField = new JTextField();  
paidField.setBounds(600, 140, 200, 30);  
mainPanel.add(paidField);
```

```
JLabel removLabel = new JLabel("Removal Reason:");
```

```
removLabel.setBounds(440, 180, 150, 30);  
mainPanel.add(removLabel);
```

```
removalField = new JTextField();  
removalField.setBounds(600, 180, 200, 30);  
mainPanel.add(removalField);
```

```
JLabel trainerLabel = new JLabel("Trainer Name:");  
trainerLabel.setBounds(440, 220, 150, 30);  
mainPanel.add(trainerLabel);
```

```
trainerField = new JTextField();  
trainerField.setBounds(600, 220, 200, 30);  
mainPanel.add(trainerField);
```

```
JLabel planLabel = new JLabel("Membership Plan:");  
planLabel.setBounds(440, 260, 150, 30);  
mainPanel.add(planLabel);
```

```
String[] plan = {"basic", "standard", "deluxe"};  
planBox = new JComboBox<>(plan);  
planBox.setBounds(600, 260, 200, 25);  
planBox.setBackground(Color.WHITE);  
mainPanel.add(planBox);
```

```
JLabel discountLabel = new JLabel("Discount Amount:");  
discountLabel.setBounds(440, 300, 150, 30);  
mainPanel.add(discountLabel);
```

```
disAmouField = new JTextField();  
disAmouField.setBounds(600, 300, 200, 30);  
disAmouField.setEditable(false);  
disAmouField.setEnabled(false);  
mainPanel.add(disAmouField);
```

```
JLabel regularPlanPrice = new JLabel("Regular Plan Price");  
regularPlanPrice.setBounds(440, 340, 150, 30);  
mainPanel.add(regularPlanPrice);
```

```
reguPriceField = new JTextField();  
reguPriceField.setBounds(600, 340, 200, 30);  
reguPriceField.setText("6500");  
reguPriceField.setEditable(false);  
reguPriceField.setEnabled(false);  
mainPanel.add(reguPriceField);
```

```
JLabel premiumPlanPrice = new JLabel("Premium Plan Price");  
premiumPlanPrice.setBounds(440, 380, 150, 30);  
mainPanel.add(premiumPlanPrice);
```

```
preCharField = new JTextField();  
preCharField.setBounds(600, 380, 200, 30);  
preCharField.setText("50000");  
preCharField.setEditable(false);  
preCharField.setEnabled(false);  
mainPanel.add(preCharField);
```

```
JPanel buttonPanel = new JPanel();
buttonPanel.setLayout(null);
buttonPanel.setBounds(30, 500, 820, 400);
frame.add(buttonPanel);

regularmembtn = new JButton("Add Regular Member");
regularmembtn.setBounds(30, 30, 200, 30);
regularmembtn.setBackground(headerColor);
regularmembtn.setForeground(Color.WHITE);
regularmembtn.setFont(new Font("Poppins", Font.PLAIN, 14));
buttonPanel.add(regularmembtn);

premiumbtn = new JButton("Add Premium Member");
premiumbtn.setBounds(300, 30, 200, 30);
premiumbtn.setBackground(headerColor);
premiumbtn.setForeground(Color.WHITE);
premiumbtn.setFont(new Font("Poppins", Font.PLAIN, 14));
buttonPanel.add(premiumbtn);

regularmembtn.addActionListener(new ActionListener() {
    @Override
    public void actionPerformed(ActionEvent e) {
        addRegularMember();
    }
});

premiumbtn.addActionListener(new ActionListener() {
```



```
@Override
public void actionPerformed(ActionEvent e) {
    addPremiumMember();
}
});

activateMembershipbtn = new JButton("Activate Membership");
activateMembershipbtn.setBounds(600, 30, 200, 30);
activateMembershipbtn.setBackground(headerColor);
activateMembershipbtn.setForeground(Color.WHITE);
activateMembershipbtn.setFont(new Font("Poppins", Font.PLAIN, 14));
buttonPanel.add(activateMembershipbtn);

activateMembershipbtn.addActionListener(new ActionListener() {
    @Override
    public void actionPerformed(ActionEvent e) {
        addactivateMembership();
    }
});

deactivateMembershipButtton = new JButton("Deactivate Membership");
deactivateMembershipButtton.setBounds(30, 90, 200, 30);
deactivateMembershipButtton.setBackground(headerColor);
deactivateMembershipButtton.setForeground(Color.WHITE);
deactivateMembershipButtton.setFont(new Font("Poppins", Font.PLAIN, 14));
buttonPanel.add(deactivateMembershipButtton);

deactivateMembershipButtton.addActionListener(new ActionListener() {
```

```
@Override  
public void actionPerformed(ActionEvent e) {  
    adddeactiveMembershipButtton();  
}  
});
```

```
markAttendancebtn = new JButton("Mark Attendance");  
markAttendancebtn.setBounds(300, 90, 200, 30);  
markAttendancebtn.setBackground(headerColor);  
markAttendancebtn.setForeground(Color.WHITE);  
markAttendancebtn.setFont(new Font("Poppins", Font.PLAIN, 14));  
buttonPanel.add(markAttendancebtn);
```

```
markAttendancebtn.addActionListener(new ActionListener() {  
    @Override  
    public void actionPerformed(ActionEvent e) {  
        addmarkAttendancebtn();  
    }  
});
```

```
revertRegularMemberbtn = new JButton("Revert Regular Member");  
revertRegularMemberbtn.setBounds(600, 90, 200, 30);  
revertRegularMemberbtn.setBackground(headerColor);  
revertRegularMemberbtn.setForeground(Color.WHITE);  
revertRegularMemberbtn.setFont(new Font("Poppins", Font.PLAIN, 14));  
buttonPanel.add(revertRegularMemberbtn);
```

```
revertRegularMemberbtn.addActionListener(new ActionListener() {
```

```
@Override
public void actionPerformed(ActionEvent e) {
    addrevertRegularMemberbtn();
}
});

revertPremiumbtn = new JButton("Revert Premium Member");
revertPremiumbtn.setBounds(30, 150, 200, 30);
revertPremiumbtn.setBackground(headerColor);
revertPremiumbtn.setForeground(Color.WHITE);
revertPremiumbtn.setFont(new Font("Poppins", Font.PLAIN, 14));
buttonPanel.add(revertPremiumbtn);
```

```
revertPremiumbtn.addActionListener(new ActionListener() {
    @Override
    public void actionPerformed(ActionEvent e) {
        addrevertPremiumMemberbtn();
    }
});
```

```
JButton displayMembersButton = new JButton("Display Members");
displayMembersButton.setBounds(300, 150, 200, 30);
displayMembersButton.setBackground(headerColor);
displayMembersButton.setForeground(Color.WHITE);
displayMembersButton.setFont(new Font("Poppins", Font.PLAIN, 14));
buttonPanel.add(displayMembersButton);
```

```
displayMembersButton.addActionListener(new ActionListener() {
```

```
@Override
public void actionPerformed(ActionEvent e) {
    displayMembers();
}
});

JButton clearFieldsButton = new JButton("Clear Fields");
clearFieldsButton.setBounds(600, 150, 200, 30);
clearFieldsButton.setBackground(headerColor);
clearFieldsButton.setForeground(Color.WHITE);
clearFieldsButton.setFont(new Font("Poppins", Font.PLAIN, 14));
buttonPanel.add(clearFieldsButton);

clearFieldsButton.addActionListener(new ActionListener() {
    @Override
    public void actionPerformed(ActionEvent e) {
        clearFields();
    }
});

discountButton = new JButton("Calculate Discount");
discountButton.setBounds(30, 210, 200, 30);
discountButton.setBackground(headerColor);
discountButton.setForeground(Color.WHITE);
discountButton.setFont(new Font("Poppins", Font.PLAIN, 14));
buttonPanel.add(discountButton);

discountButton.addActionListener(new ActionListener() {
```

```
@Override
public void actionPerformed(ActionEvent e) {
    calculateDiscount();
}
});

upgradeButton = new JButton("Upgrade Plan");
upgradeButton.setBounds(300, 210, 200, 30);
upgradeButton.setBackground(headerColor);
upgradeButton.setForeground(Color.WHITE);
upgradeButton.setFont(new Font("Poppins", Font.PLAIN, 14));
buttonPanel.add(upgradeButton);

upgradeButton.addActionListener(new ActionListener() {
    @Override
    public void actionPerformed(ActionEvent e) {
        addUpgradePlan();
    }
});

JButton payDueAmountButton = new JButton("Pay Due Amount");
payDueAmountButton.setBounds(600, 210, 200, 30);
payDueAmountButton.setBackground(headerColor);
payDueAmountButton.setForeground(Color.WHITE);
payDueAmountButton.setFont(new Font("Poppins", Font.PLAIN, 14));
buttonPanel.add(payDueAmountButton);

payDueAmountButton.addActionListener(new ActionListener() {
```

```
@Override
public void actionPerformed(ActionEvent e) {
    addPayDueAmount();
}
});

JButton savetoFileButton = new JButton("Save to File");
savetoFileButton.setBounds(30, 270, 200, 30);
savetoFileButton.setBackground(headerColor);
savetoFileButton.setForeground(Color.WHITE);
savetoFileButton.setFont(new Font("Poppins", Font.PLAIN, 14));
buttonPanel.add(savetoFileButton);

savetoFileButton.addActionListener(new ActionListener() {
    @Override
    public void actionPerformed(ActionEvent e) {
        saveToFile();
    }
});

JButton readFromFile = new JButton("Read from File");
readFromFile.setBounds(300, 270, 200, 30);
readFromFile.setBackground(headerColor);
readFromFile.setForeground(Color.WHITE);
readFromFile.setFont(new Font("Poppins", Font.PLAIN, 14));
buttonPanel.add(readFromFile);

readFromFile.addActionListener(new ActionListener() {
```

```
@Override
public void actionPerformed(ActionEvent e) {
    readFromFile();
}

});

frame.setVisible(true);
}

/**
 * Generates an array of years for date selection (1975-2025).
 * @return String array containing year values
 */
private String[] getYears() {
    String[] years = new String[50];
    for (int i = 0; i < 50; i++) {
        years[i] = String.valueOf(2025 - i);
    }
    return years;
}

/**
 * Generates an array of months for date selection (1-12).
 * @return String array containing month values
 */
private String[] getMonths() {
    String[] months = new String[12];
    for (int i = 0; i < 12; i++) {
```

```
        months[i] = String.valueOf(i + 1);
    }
    return months;
}

/**
 * Generates an array of days for date selection (1-31).
 * @return String array containing day values
 */
public String[] getDays() {
    String[] days = new String[31];
    for (int i = 0; i < 31; i++) {
        days[i] = String.valueOf(i + 1);
    }
    return days;
}

/**
 * Checks if a member ID already exists.
 * @param id The ID to check
 * @return The GymMember if found, null otherwise
 */
private GymMember isValidId(int id) {
    for (GymMember member : members) {
        if (member.getId() == id) {
            return member;
        }
    }
}
```



```
        return null;
    }

    /**
     * Adds a new regular member to the system after validating input fields.
     * Collects personal information, membership details, and referral source.
     * Displays error messages for invalid inputs or duplicate IDs.
     */
    public void addRegularMember() {
        try {
            int id = Integer.parseInt(idField.getText());
            if (isValidId(id) != null) {
                JOptionPane.showMessageDialog(frame, "The id already exists, please
enter another id.", "Warning", JOptionPane.WARNING_MESSAGE);
                return;
            }

            String name = nameField.getText();
            String location = locationField.getText();
            String phone = phoneField.getText();
            String email = emailField.getText();
            String referral = referralField.getText();

            String gender = "";
            if (maleButton.isSelected()) {
                gender = "male";
            } else if (femaleButton.isSelected()) {
                gender = "female";
            } else if (otherButton.isSelected()) {
```

```
        gender = "other";
    } else {
        JOptionPane.showMessageDialog(frame, "Please select a gender!",
            "Error", JOptionPane.ERROR_MESSAGE);
        return;
    }

    if (dobYearBox.getSelectedIndex() == -1 || dobMonthBox.getSelectedIndex()
        == -1 || dobDayBox.getSelectedIndex() == -1 ||
        msYearBox.getSelectedIndex() == -1 || msMonthBox.getSelectedIndex()
        == -1 || msDayBox.getSelectedIndex() == -1) {
        JOptionPane.showMessageDialog(frame, "Please select valid dates for
            DOB and Membership Start!", "Error", JOptionPane.ERROR_MESSAGE);
        return;
    }

    String DOB = dobYearBox.getSelectedItem() + "-" +
        dobMonthBox.getSelectedItem() + "-" + dobDayBox.getSelectedItem();

    String membershipStartDate = msYearBox.getSelectedItem() + "-" +
        msMonthBox.getSelectedItem() + "-" + msDayBox.getSelectedItem();

    if (name.isEmpty() || location.isEmpty() || phone.isEmpty() || email.isEmpty()
        || referral.isEmpty()) {
        JOptionPane.showMessageDialog(frame, "Please fill all the required
            fields!", "Error", JOptionPane.ERROR_MESSAGE);
        return;
    }

    if (!phone.matches("\\d{10}")) {
        JOptionPane.showMessageDialog(frame, "Phone number must be exactly
            10 digits (e.g., 1234567890)!", "Error", JOptionPane.ERROR_MESSAGE);
        return;
    }
}
```

```

    }

    if (!email.matches("^[\\w._%+-]+@[\\w.-]+\\.[a-zA-Z]{2,}$")) {
        JOptionPane.showMessageDialog(frame, "Please enter a valid email
address (e.g., example@domain.com)!", "Error", JOptionPane.ERROR_MESSAGE);
        return;
    }

    RegularMember member = new RegularMember(id, name, location, phone,
email, gender, DOB, membershipStartDate, referral);
    members.add(member);

    JOptionPane.showMessageDialog(frame, "Successfully added to Regular
Member!", "Success", JOptionPane.INFORMATION_MESSAGE);

} catch (NumberFormatException nfe) {
    JOptionPane.showMessageDialog(frame, "Please enter a valid ID number!",
"Error", JOptionPane.ERROR_MESSAGE);
}
}

/**
 * Adds a new premium member to the system with personal trainer information.
 * Validates required fields and checks for duplicate IDs.
 * Shows error messages for incomplete data or invalid inputs.
 */
public void addPremiumMember() {
    try {
        int id = Integer.parseInt(idField.getText());
        if (isValidId(id) != null) {

```

```
JOptionPane.showMessageDialog(frame, "Duplicate ID exists. Please  
enter a unique ID.", "Error", JOptionPane.ERROR_MESSAGE);
```

```
    return;
```

```
}
```

```
String name = nameField.getText();
```

```
String location = locationField.getText();
```

```
String phone = phoneField.getText();
```

```
String email = emailField.getText();
```

```
String trainer = trainerField.getText();
```

```
String gender = "";
```

```
if (maleButton.isSelected()) {
```

```
    gender = "male";
```

```
} else if (femaleButton.isSelected()) {
```

```
    gender = "female";
```

```
} else if (otherButton.isSelected()) {
```

```
    gender = "other";
```

```
} else {
```

```
    JOptionPane.showMessageDialog(frame, "Please select a gender!",  
"Error", JOptionPane.ERROR_MESSAGE);
```

```
    return;
```

```
}
```

```
if (dobYearBox.getSelectedIndex() == -1 || dobMonthBox.getSelectedIndex()  
== -1 || dobDayBox.getSelectedIndex() == -1 ||
```

```
    msYearBox.getSelectedIndex() == -1 || msMonthBox.getSelectedIndex()  
== -1 || msDayBox.getSelectedIndex() == -1) {
```

```
    JOptionPane.showMessageDialog(frame, "Please select valid dates for  
DOB and Membership Start!", "Error", JOptionPane.ERROR_MESSAGE);
```

```
        return;
    }

    String DOB = dobYearBox.getSelectedItem() + "-" +
dobMonthBox.getSelectedItem() + "-" + dobDayBox.getSelectedItem();

    String membershipStartDate = msYearBox.getSelectedItem() + "-" +
msMonthBox.getSelectedItem() + "-" + msDayBox.getSelectedItem();

    if (name.isEmpty() || location.isEmpty() || phone.isEmpty() || email.isEmpty()
|| trainer.isEmpty()) {
        JOptionPane.showMessageDialog(frame, "Please fill all the required
fields!", "Error", JOptionPane.ERROR_MESSAGE);
        return;
    }

    if (!phone.matches("\\d{10}")) {
        JOptionPane.showMessageDialog(frame, "Phone number must be exactly
10 digits (e.g., 1234567890)!", "Error", JOptionPane.ERROR_MESSAGE);
        return;
    }

    if (!email.matches("^\\w._%+~]+@[\\w.-]+\\.\\w+[a-zA-Z]{2,}$")) {
        JOptionPane.showMessageDialog(frame, "Please enter a valid email
address (e.g., example@domain.com)!", "Error", JOptionPane.ERROR_MESSAGE);
        return;
    }

    PremiumMember member = new PremiumMember(id, name, location,
phone, email, gender, DOB, membershipStartDate, trainer);
    members.add(member);

    JOptionPane.showMessageDialog(frame, "Successfully added to Premium
Members!", "Success", JOptionPane.INFORMATION_MESSAGE);
```

```
    } catch (NumberFormatException efc) {
        JOptionPane.showMessageDialog(frame, "Please enter a valid ID!", "Error",
JOptionPane.ERROR_MESSAGE);
    }
}

/**
 * Activates a member's membership using their ID.
 * Shows error if member not found or invalid ID format is entered.
 */
public void addactivateMembership() {
    try {
        int id = Integer.parseInt(idField.getText());
        GymMember member = isValidId(id);
        if (member == null) {
            JOptionPane.showMessageDialog(frame, "Member not found!", "Error",
JOptionPane.ERROR_MESSAGE);
            return;
        }
        member.activateMembership();
        JOptionPane.showMessageDialog(frame, "Membership activated
successfully!", "Success", JOptionPane.INFORMATION_MESSAGE);

    } catch (NumberFormatException efc) {
        JOptionPane.showMessageDialog(frame, "Invalid ID format! Please enter a
number.", "Error", JOptionPane.ERROR_MESSAGE);
    }
}
```

```
/**
 * Deactivates a member's membership using their ID.
 * Displays error messages for invalid ID or non-existent members.
 */
public void adddeactiveMembershipButtton() {
    try {
        int id = Integer.parseInt(idField.getText());
        GymMember member = isValidId(id);
        if (member == null) {
            JOptionPane.showMessageDialog(frame, "Member not found!", "Error",
JOptionPane.ERROR_MESSAGE);
            return;
        }
        member.deactivateMembership();
        JOptionPane.showMessageDialog(frame, "Membership deactivated
successfully!", "Success", JOptionPane.INFORMATION_MESSAGE);
    } catch (NumberFormatException efc) {
        JOptionPane.showMessageDialog(frame, "Invalid ID format! Please enter a
number.", "Error", JOptionPane.ERROR_MESSAGE);
    }
}

/**
 * Marks attendance for a member if their membership is active.
 * Requires valid member ID and active membership status.
 */
public void addmarkAttendancebtn() {
    try {
```

```
        int id = Integer.parseInt(idField.getText());
        GymMember member = isValidId(id);
        if (member == null) {
            JOptionPane.showMessageDialog(frame, "Member not found!", "Error",
JOptionPane.ERROR_MESSAGE);
            return;
        }
        if (!member.getActiveStatus()) {
            JOptionPane.showMessageDialog(frame, "Member is not active!", "Error",
JOptionPane.ERROR_MESSAGE);
            return;
        }
        member.markAttendance();
        JOptionPane.showMessageDialog(frame, "Attendance marked
successfully!", "Success", JOptionPane.INFORMATION_MESSAGE);
    } catch (NumberFormatException efc) {
        JOptionPane.showMessageDialog(frame, "Invalid ID format! Please enter a
number.", "Error", JOptionPane.ERROR_MESSAGE);
    }
}

/**
 * Reverts a regular member's status with a removal reason.
 * Checks member type and validates removal reason input.
 */
public void addrevertRegularMemberbtn() {
    try {
        int id = Integer.parseInt(idField.getText());
        GymMember member = isValidId(id);
        if (member == null) {
```



```

        JOptionPane.showMessageDialog(frame, "Member not found!", "Error",
JOptionPane.ERROR_MESSAGE);
        return;
    }
    if (!(member instanceof RegularMember)) {
        JOptionPane.showMessageDialog(frame, "This member is not a regular
member!", "Error", JOptionPane.ERROR_MESSAGE);
        return;
    }
    String reason = removalField.getText();
    if (reason.isEmpty()) {
        JOptionPane.showMessageDialog(frame, "Please provide a removal
reason!", "Error", JOptionPane.ERROR_MESSAGE);
        return;
    }
    RegularMember regularMember = (RegularMember) member;
    regularMember.revertRegularMember(reason);
    JOptionPane.showMessageDialog(frame, "Regular member reverted
successfully!", "Success", JOptionPane.INFORMATION_MESSAGE);
} catch (NumberFormatException efc) {
    JOptionPane.showMessageDialog(frame, "Invalid ID format! Please enter a
number.", "Error", JOptionPane.ERROR_MESSAGE);
}
}

/**
 * Reverts a premium member's status with a removal reason.
 * Checks member type and validates removal reason input.
 */
public void addrevertPremiumMemberbtn() {

```

```
try {
    int id = Integer.parseInt(idField.getText());
    GymMember member = isValidId(id);
    if (member == null) {
        JOptionPane.showMessageDialog(frame, "Member not found!", "Error",
JOptionPane.ERROR_MESSAGE);
        return;
    }
    if (!(member instanceof PremiumMember)) {
        JOptionPane.showMessageDialog(frame, "This member is not a premium
member!", "Error", JOptionPane.ERROR_MESSAGE);
        return;
    }
    String reason = removalField.getText();
    if (reason.isEmpty()) {
        JOptionPane.showMessageDialog(frame, "Please provide a removal
reason!", "Error", JOptionPane.ERROR_MESSAGE);
        return;
    }
    PremiumMember premiumMember = (PremiumMember) member;
    premiumMember.revertPremiumMember();
    JOptionPane.showMessageDialog(frame, "Premium member reverted
successfully!", "Success", JOptionPane.INFORMATION_MESSAGE);
} catch (NumberFormatException efc) {
    JOptionPane.showMessageDialog(frame, "Invalid ID format! Please enter a
number.", "Error", JOptionPane.ERROR_MESSAGE);
}
}

/**
```

- * Displays all members in a scrollable text area with detailed information.
- * Shows different details for regular and premium members including:
 - * - Membership status
 - * - Attendance records
 - * - Payment information
 - * - Loyalty points
- */

```
public void displayMembers() {  
    JFrame displayFrame = new JFrame("Member List");  
    displayFrame.setDefaultCloseOperation(JFrame.DISPOSE_ON_CLOSE);  
    displayFrame.setSize(600, 600);  
    displayFrame.setLayout(null);  
  
    JTextArea displayArea = new JTextArea();  
    displayArea.setEditable(false);  
    JScrollPane scrollPane = new JScrollPane(displayArea);  
    scrollPane.setBounds(10, 10, 560, 540);  
    displayFrame.add(scrollPane);  
  
    if (members.isEmpty()) {  
        displayArea.setText("No members found!");  
    } else {  
        StringBuilder displayText = new StringBuilder();  
        for (GymMember member : members) {  
            if (member instanceof RegularMember) {  
                displayText.append("--- REGULAR MEMBER ---\n");  
            } else if (member instanceof PremiumMember) {  
                displayText.append("--- PREMIUM MEMBER ---\n");  
            }  
        }  
    }  
}
```

```
    }  
    displayText.append("ID: ").append(member.getId()).append("\n");  
    displayText.append("Name: ").append(member.getName()).append("\n");  
    displayText.append("Location:  
").append(member.getLocation()).append("\n");  
    displayText.append("Phone: ").append(member.getPhone()).append("\n");  
    displayText.append("Email: ").append(member.getEmail()).append("\n");  
    displayText.append("Gender:  
").append(member.getGender()).append("\n");  
    displayText.append("DOB: ").append(member.getDOB()).append("\n");  
    displayText.append("Start Date:  
").append(member.getMembershipStartDate()).append("\n");  
    displayText.append("Attendance:  
").append(member.getAttendance()).append("\n");  
    displayText.append("Loyalty Points:  
").append(member.getLoyaltyPoints()).append("\n");  
    displayText.append("Active Status:  
").append(member.getActiveStatus()).append("\n");  
  
    if (member instanceof RegularMember) {  
        RegularMember regMember = (RegularMember) member;  
        displayText.append("Plan:  
").append(regMember.getPlan()).append("\n");  
        displayText.append("Price:  
").append(regMember.getPrice()).append("\n");  
        String removalReason = regMember.getRemovalReason();  
        if (removalReason != null && !removalReason.isEmpty()) {  
            displayText.append("Removal Reason:  
").append(removalReason).append("\n");  
        }  
    } else if (member instanceof PremiumMember) {
```

```

        PremiumMember premMember = (PremiumMember) member;

        displayText.append("Personal Trainer:
").append(premMember.getPersonalTrainer()).append("\n");

        displayText.append("Paid Amount:
").append(premMember.getPaidAmount()).append("\n");

        displayText.append("Payment Status:
").append(premMember.getIsFullPayment() ? "Complete" :
"Incomplete").append("\n");

        double remainingAmount = premMember.getPremiumCharge() -
premMember.getPaidAmount();

        displayText.append("Remaining Amount:
").append(remainingAmount).append("\n");

        if (premMember.getIsFullPayment()) {
            displayText.append("Discount Amount:
").append(premMember.getDiscountAmount()).append("\n");
        }
    }

    displayText.append("\n-----\n\n");
}

displayArea.setText(displayText.toString());
}

displayFrame.setVisible(true);
}

/**
 * Clears all input fields in the GUI.
 */
private void clearFields() {
    nameField.setText("");
    locationField.setText("");
}

```

```
phoneField.setText("");
emailField.setText("");
idField.setText("");
dobYearBox.setSelectedIndex(-1);
dobMonthBox.setSelectedIndex(-1);
dobDayBox.setSelectedIndex(-1);
msYearBox.setSelectedIndex(-1);
msMonthBox.setSelectedIndex(-1);
msDayBox.setSelectedIndex(-1);
disAmouField.setText("");
referralField.setText("");
paidField.setText("");
removalField.setText("");
trainerField.setText("");
genderGroup.clearSelection();
planBox.setSelectedIndex(-1);
}
```

```
/**
```

```
 * Calculates and displays discount amount for premium members who made full
 payment.
```

```
 * Shows error for non-premium members or incomplete payments.
```

```
 */
```

```
public void calculateDiscount() {
    try {
        int id = Integer.parseInt(idField.getText());
        GymMember member = isValidId(id);

        if (member == null) {
```

```
        JOptionPane.showMessageDialog(frame, "Member not found!", "Error",
        JOptionPane.ERROR_MESSAGE);
        return;
    }
```

```
    if (!(member instanceof PremiumMember)) {
        JOptionPane.showMessageDialog(frame, "Only premium members can
        calculate discount!", "Error", JOptionPane.ERROR_MESSAGE);
        return;
    }
```

```
    PremiumMember premiumMember = (PremiumMember) member;
    try {
        double premiumCharge = Double.parseDouble(preCharField.getText());
        premiumMember.payDueAmount(premiumCharge);
        premiumMember.calculateDiscount();

        if (premiumMember.getIsFullPayment()) {

            disAmouField.setText(String.valueOf(premiumMember.getDiscountAmount()));
        } else {
            disAmouField.setText("0");
        }
    } catch (NumberFormatException e) {
        JOptionPane.showMessageDialog(frame, "Invalid premium charge
        amount!", "Error", JOptionPane.ERROR_MESSAGE);
        return;
    }
} catch (NumberFormatException e) {
```

```
JOptionPane.showMessageDialog(frame, "Invalid ID format! Please enter a number.", "Error", JOptionPane.ERROR_MESSAGE);
```

```
}
```

```
}
```

```
/**
```

```
 * Upgrades a regular member's plan based on attendance and current plan.
```

```
 * Validates attendance requirements and active membership status.
```

```
 * Updates plan price display after successful upgrade.
```

```
 */
```

```
public void addUpgradePlan() {
```

```
    try {
```

```
        int id = Integer.parseInt(idField.getText());
```

```
        GymMember member = isValidId(id);
```

```
        if (member == null) {
```

```
            JOptionPane.showMessageDialog(frame, "Member not found!", "Error",  
JOptionPane.ERROR_MESSAGE);
```

```
            return;
```

```
        }
```

```
        if (!(member instanceof RegularMember)) {
```

```
            JOptionPane.showMessageDialog(frame, "Only regular members can  
upgrade plans!", "Error", JOptionPane.ERROR_MESSAGE);
```

```
            return;
```

```
        }
```

```
        if (!member.getActiveStatus()) {
```

```
            JOptionPane.showMessageDialog(frame, "Membership is not active!",  
"Error", JOptionPane.ERROR_MESSAGE);
```



```
        return;
    }

    RegularMember regularMember = (RegularMember) member;

    if (regularMember.getAttendance() < regularMember.getAttendanceLimit()) {
        JOptionPane.showMessageDialog(frame,
            "Member needs " + (regularMember.getAttendanceLimit() -
            regularMember.getAttendance()) +
            " more visits to be eligible for upgrade",
            "Not Eligible", JOptionPane.WARNING_MESSAGE);
        return;
    }

    String newPlan = (String) planBox.getSelectedItem();

    if (newPlan == null || newPlan.equalsIgnoreCase(regularMember.getPlan()))
    {
        JOptionPane.showMessageDialog(frame,
            "Member is already on " + regularMember.getPlan() + " plan!",
            "Same Plan", JOptionPane.INFORMATION_MESSAGE);
        return;
    }

    String result = regularMember.upgradePlan(newPlan);
    JOptionPane.showMessageDialog(frame, result, "Upgrade Status",
    JOptionPane.INFORMATION_MESSAGE);

    reguPriceField.setText(String.valueOf(regularMember.getPrice()));
```

```
    } catch (NumberFormatException e) {  
        JOptionPane.showMessageDialog(frame, "Invalid ID format! Please enter a  
number.", "Error", JOptionPane.ERROR_MESSAGE);  
    }  
}  
  
/**  
 * Processes payment of due amount for premium members.  
 */  
public void addPayDueAmount() {  
    try {  
        int id = Integer.parseInt(idField.getText());  
        GymMember member = isValidId(id);  
        if (member == null) {  
            JOptionPane.showMessageDialog(frame, "Member not found!", "Error",  
JOptionPane.ERROR_MESSAGE);  
            return;  
        }  
        if (!(member instanceof PremiumMember)) {  
            JOptionPane.showMessageDialog(frame, "Only premium members can  
pay due amount!", "Error", JOptionPane.ERROR_MESSAGE);  
            return;  
        }  
        PremiumMember premiumMember = (PremiumMember) member;  
        try {  
            double premiumCharge = Double.parseDouble(preCharField.getText());  
            premiumMember.payDueAmount(premiumCharge);  
            JOptionPane.showMessageDialog(frame, "Successfully paid the due  
amount!", "Success", JOptionPane.INFORMATION_MESSAGE);  
        }  
    }  
}
```

```

    } catch (NumberFormatException e) {
        JOptionPane.showMessageDialog(frame, "Invalid premium charge
amount!", "Error", JOptionPane.ERROR_MESSAGE);
        return;
    }
} catch (NumberFormatException efx) {
    JOptionPane.showMessageDialog(frame, "Invalid ID format! Please enter a
number.", "Error", JOptionPane.ERROR_MESSAGE);
}
}

/**
 * Saves member details to a text file.
 */
private void saveToFile() {
    try (FileWriter writer = new FileWriter("MembersDetails.txt")) {
        writer.write("Gym Member Details\n");
        writer.write(String.format("%-5s %-15s %-15s %-15s %-25s %-20s %-10s %-
10s %-10s %-10s %-15s %-10s %-15s %-15s %-15s\n",
            "ID", "Name", "Location", "Phone", "Email", "Membership Start Date",
            "Plan",
            "Price", "Attendance", "Loyalty Points", "Active Status", "Full Payment",
            "Discount Amount", "Paid Amount", "Net Amount Paid"));

        for (GymMember member : members) {
            if (member instanceof RegularMember) {
                RegularMember regular = (RegularMember) member;
                writer.write(String.format("%-5d %-15s %-15s %-15s %-25s %-20s %-
10s %-10.2f %-10d %-10.2f %-15b %-10s %-15s %-15s %-15s\n",

```

```

        regular.getId(), regular.getName(), regular.getLocation(),
        regular.getPhone(),
        regular.getEmail(), regular.getMembershipStartDate(),
        regular.getPlan(),
        regular.getPrice(), regular.getAttendance(),
        regular.getLoyaltyPoints(),
        regular.getActiveStatus(), "N/A", "N/A", "N/A", "N/A"));
    } else if (member instanceof PremiumMember) {
        PremiumMember premium = (PremiumMember) member;
        writer.write(String.format("%-5d %-15s %-15s %-15s %-25s %-20s %-10s %-10s %-10d %-10.2f %-15b %-10b %-15.2f %-15.2f %-15.2f\n",
        premium.getId(), premium.getName(), premium.getLocation(),
        premium.getPhone(),
        premium.getEmail(), premium.getMembershipStartDate(), "Premium",
        "N/A", premium.getAttendance(), premium.getLoyaltyPoints(),
        premium.getActiveStatus(), premium.getIsFullPayment(),
        premium.getDiscountAmount(),
        premium.getPaidAmount(), (premium.getPremiumCharge() -
        premium.getPaidAmount())));
    }
}

JOptionPane.showMessageDialog(frame, "Members saved to file
successfully!", "Success", JOptionPane.INFORMATION_MESSAGE);

} catch (IOException e) {

    JOptionPane.showMessageDialog(frame, "Error saving file: " +
    e.getMessage(), "Error", JOptionPane.ERROR_MESSAGE);

}

}

/**
 * Reads member details from a text file and displays them.

```

```
*/  
  
public void readFromFile() {  
    JTextArea textArea = new JTextArea(20, 80);  
    textArea.setEditable(false);  
    JScrollPane scrollPane = new JScrollPane(textArea);  
  
    try (Scanner scanner = new Scanner(new File("MembersDetails.txt"))) {  
        while (scanner.hasNextLine()) {  
            textArea.append(scanner.nextLine() + "\n");  
        }  
        JOptionPane.showMessageDialog(frame, scrollPane, "Member Details",  
JOptionPane.INFORMATION_MESSAGE);  
    } catch (IOException e) {  
        JOptionPane.showMessageDialog(frame, "Error reading file: " +  
e.getMessage(), "Error", JOptionPane.ERROR_MESSAGE);  
    }  
}  
  
public static void main(String[] args) {  
    new GymGUI();  
}  
}
```